Design and Analysis of Algorithms

Review Notes – Spring 2025

What is an Algorithm?

An algorithm is a sequence of well-defined steps to solve a problem. For example, to find the maximum element in an array:

- Initialize a candidate maximum as the first element.
- Compare each element, updating the candidate if a larger element is found.
- Return the candidate maximum.

Algorithm Design Principles

When designing an algorithm, always address the following:

- **What problem are we solving?** Clearly define the input and desired output.
- **How does the algorithm work?** Provide pseudocode or a clear explanation of steps.
- **Why does it work?** Prove correctness using techniques like induction, contradiction, or the exchange argument.
- **How efficient is it?** Analyze its runtime, space complexity, and other resource usage.

For example, in designing a sorting algorithm:

- Define the input as an array and the output as a permutation of the array in non-decreasing order.
- Use a step-by-step explanation or pseudocode (e.g., Merge Sort or Bubble Sort).
- Prove that the algorithm always sorts the array correctly (e.g., show the merging step in Merge Sort maintains order).
- Analyze runtime: Merge Sort runs in $O(n \log n)$, while Bubble Sort is $O(n^2)$.

Proof of Correctness

Every algorithm requires a rigorous proof of correctness to ensure it works for all inputs:

- **Induction/Recursion:** Prove the base case, then correctness of larger inputs follows from smaller inputs because of the recursive structure.
- **Contradiction:** Assume failure and show inconsistency.
- **Exchange Argument:** Demonstrate that any valid solution can be transformed into the one proposed by your algorithm without loss of optimality (e.g., correctness of Huffman Encoding or Greedy Scheduling).

Runtime Analysis and the RAM Model

Efficiency is a key consideration in algorithms, and runtime is typically analyzed using the Random Access Machine (RAM) model:

- **Basic Operations:** Simple tasks like arithmetic (e.g., addition, multiplication), comparisons (e.g., `if-statements`), and memory access (e.g., reading or writing an array element) are assumed to take constant time, $O(1)$.
- **Hardware Independence:** The RAM model abstracts away hardware details such as CPU speed or memory latency, focusing instead on the input size and algorithm structure.
- **Input Size:** Runtime is expressed as a function of the size of the input (e.g., for an array, the size n).

Key Insight: The RAM model lets us focus on how algorithms scale with input size rather than exact runtimes on specific machines.

Asymptotic Notation

Asymptotic notation helps describe how runtime grows with input size, focusing on dominant terms while ignoring constants and lower-order terms.

- $O(f(n))$**:** Big-O gives an upper bound on growth. For example, if $T(n) = 5n^2 + 3n$, then $T(n) = O(n^2)$ because the n^2 term dominates as $n \to \infty$.
- $\Omega(f(n))$**:** Big-Omega gives a lower bound on growth. For instance, $T(n) = 2n + 1$ is $\Omega(n)$ because n is the slowest-growing term that bounds $T(n)$ from below.
- $\Theta(f(n))$**:** Theta gives a tight bound when both Big-O and Big-Omega apply. For example, $T(n) = n^2 + 3n$ is $\Theta(n^2)$.
- $o(f(n))$**:** Little-O describes functions that grow strictly slower than $f(n)$. For instance, $\log n = o(n)$ because $\log n/n \to 0$ as $n \to \infty$.
- $\omega(f(n))$**:** Little-Omega describes functions that grow strictly faster than $f(n)$. For example, $n^2 = \omega(n)$.

Analyzing Growth Rates

- **Focus on Dominant Terms:** Ignore constants and lower-order terms. For example:
$$f(n) = 5n^3 + 3n^2 + 7n + 10 \implies \Theta(n^3).$$
- **Use Limits to Compare Growth:** Compare $f(n)$ and $g(n)$ using:
$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0, & f(n) = o(g(n)); \\ c > 0, & f(n) = \Theta(g(n)); \\ \infty, & f(n) = \omega(g(n)). \end{cases}$$
Example: $\frac{n^2}{n^3} = \frac{1}{n} \to 0$, so $n^2 = o(n^3)$.

- **Logarithmic Growth:** $(\log n)^k = o(n^c)$, $c > 0$. Logs of different bases differ by a constant factor:
$$\log_b n = \frac{\log_a n}{\log_a b}.$$
- **Polynomials:** $n^a = o(n^b)$ for $a < b$. Adding terms doesn't change the bound:
$$n^a + n^b = \Theta(n^b), \quad a < b.$$
- **Exponentials:** $n^k = o(c^n)$, $k > 0, c > 1$.
- **Factorials:** $n!$ grows faster than exponential functions:
$$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \quad \text{(Stirling's approximation)}.$$
- **Iterated Growth:** For large-scale growth:
$$c^n \ll n! \ll n^n \ll n^{n^2} \ll 2^{2^n}.$$
- **Hierarchy of Growth Rates:**
$$1 \ll \log\log n \ll \log n \ll (\log n)^2 \ll n^{1/3} \ll \sqrt{n} \ll n \ll n\log n \ll n^2 \ll n^3 \ll n^k$$
$$\ll c^n \ll n! \ll n^n \ll 2^{n^2} \ll 2^{2^n}.$$

Problems to Practice:

1. Rank the following in increasing order of growth:
$$\sqrt{n}, \log n, n^{\log n}, 100n, 2^n, n!.$$
2. Determine $O, o, \Omega, \omega, \Theta$ relationships for:
   - $f(n) = n^2 - 7n, g(n) = n^3 - 10n^2$.
   - $f(n) = (\sqrt{n})^3, g(n) = n^2 - (\sqrt{n})^3$.
   - $f(n) = 2^{\log_2(n)}, g(n) = n$.
3. Simplify the summations and determine their asymptotic growth:
$$\sum_{i=1}^{n} i^2, \quad \sum_{i=1}^{n} \log i, \quad \sum_{i=1}^{n} \frac{1}{i}, \quad \sum_{i=1}^{n} i\log i.$$
4. Analyze the dominant terms and tight complexities for:
$$5 + 0.001n^3 + 0.025n, \quad 500n + 100n^{1.5} + 50n\log n, \quad n^2\log n + n(\log n)^2.$$

Recurrence Relations

Recurrence relations describe runtimes of recursive algorithms. They are essential for understanding divide-and-conquer algorithms and other recursive techniques.

Some Methods for Solving Recurrences

- **Substitution (Manual Expansion):** Expand the recurrence step by step until a pattern emerges, then solve for a closed form.
  - Example: $T(n) = T(n/2) + O(1)$.

    $$T(n) = T(n/4) + O(1) + O(1) = T(n/8) + O(1) + O(1) + O(1) = \cdots.$$

    After $\log n$ steps: $T(n) = O(\log n)$.
- **Recursion Trees:** Visualize the recurrence as a tree of recursive calls. Analyze the cost at each level and sum it up.
  - Example: $T(n) = 2T(n/2) + O(n)$ (Merge Sort).

    $$\text{Level 1: } n, \quad \text{Level 2: } n, \quad \text{Level 3: } n, \cdots.$$

    Total work: $O(n \log n)$.
- $$T(n) = T(\sqrt{n}) + 1 \implies T(2^m) = T(2^{m/2}) + 1.$$

  Rename $T(2^m)$ as $S(m)$, giving $S(m) = S(m/2) + 1$. Expanding:

  $$S(m) = S(m/2) + 1 = S(m/4) + 2 = \cdots.$$

  This has $\log m$ levels, so $S(m) = O(\log m)$. Substituting back, $T(n) = O(\log \log n)$.

$T(n) = aT(n/b) + f(n)$

For recurrences of the form $T(n) = aT(n/b) + f(n)$, the solution depends on how $f(n)$ compares to $n^{\log_b a}$, which represents the "work per level" from recursive calls.

Three Cases

- **Case 1: Increasing Series ($f(n) \ll n^{\log_b a}$)**
  - Recursive work dominates because the work decreases geometrically at each level and the problem essentially looks like the case $T(n) = aT(n/b)$.
  - $T(n) = \Theta(n^{\log_b a})$.
- **Case 2: Decreasing Series ($f(n) \gg n^{\log_b a}$)**
  - The additional work $f(n)$ dominates because the lower levels contribute more and we essentially have $T(n) = \Theta(f(n))$
  - $T(n) = \Theta(f(n))$
- **Case 3: Balanced Series ($f(n) = n^{\log_b a}$)**
  - The work at each level is roughly equal, leading to a logarithmic number of levels.
  - Solution: $T(n) = \Theta(n^{\log_b a} \log n)$.

Examples

- $T(n) = 2T(n/2) + O(n)$:
  - Divide into 2 subproblems of size $n/2$. Each level of recursion performs $O(n)$ work (splitting and merging).
  - At level 0 (root): $O(n)$.
  - At level 1: 2 subproblems of size $n/2$, total cost $2 \cdot O(n/2) = O(n)$.
  - At level 2: 4 subproblems of size $n/4$, total cost $4 \cdot O(n/4) = O(n)$.
  - Each level contributes $O(n)$, and there are $\log n$ levels.
  - Total cost: $O(n \log n)$.
- $T(n) = 3T(n/3) + O(n^2)$:
  - Divide into 3 subproblems of size $n/3$. Each level of recursion performs $O(n^2)$ work for processing.
  - At level 0 (root): $O(n^2)$.
  - At level 1: 3 subproblems of size $n/3$, total cost $3 \cdot O((n/3)^2) = O(n^2/3)$.
  - At level 2: 9 subproblems of size $n/9$, total cost $9 \cdot O((n/9)^2) = O(n^2/9)$.
  - The total cost decreases geometrically, with the root level $O(n^2)$ dominating.
  - Total cost: $O(n^2)$.
- $T(n) = 3T(n/2) + O(n)$:
  - Divide into 3 subproblems of size $n/2$. Each level of recursion performs $O(n)$ work.
  - At level 0 (root): $O(n)$.
  - At level 1: 3 subproblems of size $n/2$, each contributing $O(n/2)$. Total cost:
    $$3 \cdot O(n/2) = O(3n/2).$$
  - At level 2: 9 subproblems of size $n/4$, each contributing $O(n/4)$. Total cost:
    $$9 \cdot O(n/4) = O(9n/4).$$
  - The cost increases geometrically as you go deeper in the recursion tree:
    $$O(n) + O(3n/2) + O(9n/4) + \cdots .$$
  - The total cost follows a geometric progression with ratio $3/2 > 1$, and the work grows as you move deeper into the tree.
  - The total cost is dominated by the last level. If the depth of the tree is $\log_2 n$, the work at the deepest level is:
    $$O(3^{\log_2 n}) = O(n^{\log_2 3}).$$
  - Total cost: $O(n^{\log_2 3})$.

Unbalanced Recursion Trees

Case 1: Constant Work Per Level

Example: $T(n) = T(3n/4) + T(n/4) + O(n)$

- Work per level: $O(n)$, as each subproblem contributes $O(3n/4 + n/4) = n$.
- Total levels: The height of the recursion tree is determined by the taller branch, $\max(\log_4 n, \log_{4/3} n) = O(\log n)$.
- Since work per level is constant ($O(n)$):

$$T(n) = O(n \log n).$$

Case 2: Decreasing Work Per Level

Example: $T(n) = T(n/2) + T(n/3) + O(n^2)$

- Work per level: $O(n^2)$ at the root, decreasing geometrically as we go deeper.
- Total levels: $\max(\log_2 n, \log_3 n) = O(\log n)$.
- Since work per level decreases:

$$T(n) = O(n^2).$$

Case 3: Increasing Work Per Level

Example: $T(n) = T(n/2) + T(n/3) + O(\sqrt{n})$

- **Work per level:**
  - At the root (level 0): $O(\sqrt{n})$.
  - At level 1: The subproblems contribute $O(\sqrt{n/2}) + O(\sqrt{n/3})$, giving total work:

$$O(\sqrt{n}) \cdot \left(1 + \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right).$$

    Since $\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}} > 1$, the work per level increases geometrically.
- **Tree height:**
  - The larger subproblem ($T(n/2)$) determines the height, which is $\log_2 n$.
  - Smaller height $\log_3 n$, based on $T(n/3)$, contribute lower bounds.
- **Upper Bound:** Using the larger height ($\log_2 n$), the total work is dominated by the deepest level:

$$\text{Total work} = \sum_{i=0}^{\log_2 n} O(\sqrt{n}) \cdot \left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right)^i = .O(\sqrt{n}) \left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right)^{\log_2 n} =$$

$$T(n) = O\left(n^p\right),$$

where

$$p = 0.5 + \log_2 \left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right).$$

- **Lower Bound:** Using the smaller height ($\log_3 n$), the total work is dominated by the deepest level:

$$\text{Total work} = \sum_{i=0}^{\log_3 n} O(\sqrt{n}) \cdot \left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right)^i.$$

  At the deepest level ($i = \log_3 n$), the geometric series dominates, and the total work is approximately:

$$\text{Total work} \approx O(\sqrt{n}) \cdot \left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right)^{\log_3 n}.$$

  Simplify using properties of exponents:

$$\left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right)^{\log_3 n} = \left(2^{-\frac{1}{2}} + 3^{-\frac{1}{2}}\right)^{\log_3 n}.$$

  This gives:

$$T(n) = \Omega\left(n^q\right),$$

  where:

$$q = 0.5 + \log_3\left(\frac{1}{\sqrt{2}} + \frac{1}{\sqrt{3}}\right).$$

- **Exact Growth:** Assume $T(n) = \Theta(n^p)$. Substituting into the recurrence:

$$n^p = \left(\frac{n}{2}\right)^p + \left(\frac{n}{3}\right)^p + O(\sqrt{n}),$$

  yields:

$$1 = \frac{1}{2^p} + \frac{1}{3^p}.$$

  Thus:

$$T(n) = \Theta(n^p),$$

  where p solves $1 = \frac{1}{2^p} + \frac{1}{3^p}$. $p \approx 0.7095$.

  This can be verified by induction on the following statement:

$$c_1 n^p \leq T(n) \leq c_2 n^p - c_3\sqrt{n}.$$

Recursion

Divide and Conquer

Core Idea: Break a problem into smaller parts, solve each part recursively, and combine results. Key issues:

- Why does dividing the problem often lead to faster algorithms?
- How do recursion trees help analyze runtime?

Key Algorithms:

- Binary Search
- Merge Sort
- Karatsuba's Algorithm (fast multiplication), Strassen's Matrix Multiplication
- Quick Sort
- Max Profit/Max Subarray Sum

1. Binary Search

Problem Specification

Given a sorted array $A[1 \dots n]$ of distinct elements and a target value x, determine if x exists in A.

Algorithm

The binary search algorithm works as follows:

1. Initialize two pointers: $left \leftarrow 1$ and $right \leftarrow n$
2. While $left \leq right$:
   (a) Calculate the midpoint: $mid \leftarrow \left\lfloor \frac{left+right}{2} \right\rfloor$
   (b) If $A[mid] = x$, return `Found`
   (c) If $x < A[mid]$, set $right \leftarrow mid - 1$ (search the left half)
   (d) If $x > A[mid]$, set $left \leftarrow mid + 1$ (search the right half)
3. If the loop ends without finding x, return `Not Found`

Correctness

The algorithm maintains three key properties:

- **Invariant**: At each iteration, if x exists in A, it must be within $A[left \dots right]$
- **Progress**: The search space halves each iteration (either $right$ decreases or $left$ increases)
- **Termination**: The loop terminates when $left > right$, meaning we've searched all possible locations

Runtime Analysis

The time complexity can be analyzed using the recurrence relation:

- **Recurrence**: $T(n) = T(n/2) + O(1)$
  - The $O(1)$ term accounts for the constant work at each step (comparison and pointer updates)
  - The $T(n/2)$ term comes from the recursive work on half the array
- **Solving the recurrence**:

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + c + c \\ &\vdots \\ &= T(1) + c \cdot \log_2 n \\ &= O(\log n) \end{aligned}$$

- Each comparison eliminates half of the remaining elements, the algorithm therefore runs in $\boxed{O(\log n)}$ time in the worst case.

Problem Specification

Given a unimodal array $A[1 \dots n]$ (strictly increasing then strictly decreasing), find the maximum element.

Algorithm

1. Initialize $left \leftarrow 1$, $right \leftarrow n$
2. While $left < right$:
   (a) $mid \leftarrow \left\lfloor \frac{left+right}{2} \right\rfloor$
   (b) If $A[mid] < A[mid+1]$:
      - Peak is right of mid: $left \leftarrow mid + 1$

   (c) Else:
      - Peak is at or left of mid: $right \leftarrow mid$
3. Return $A[left]$

Correctness

- **Invariant**: Maximum $\in A[left \dots right]$
- **Progress**: Always moves toward peak
- **Termination**: Converges when $left = right$

Runtime Analysis

$$
\begin{aligned}
T(n) &= T\left(\frac{n}{2}\right) + O(1) \\
&= O(\log n) \quad \text{(same as binary search)}
\end{aligned}
$$

$$
\begin{array}{c}
c \\ | \\ c \\ | \\ c \\ | \\ \vdots \\ | \\ T(1)
\end{array}
$$

2. Integer Multiplication

Problem Specification

Given two n-digit integers x and y, compute their product $x \times y$.

Algorithm 1

The standard recursive multiplication works as follows:

1. If $n = 1$, return $x \times y$ directly
2. Split each number into two halves:

$$
\begin{aligned}
x &= 10^{n/2} x_1 + x_0 \\
y &= 10^{n/2} y_1 + y_0
\end{aligned}
$$

3. Compute four products recursively:

$$
\begin{aligned}
z_{00} &= x_0 \times y_0 \\
z_{01} &= x_0 \times y_1 \\
z_{10} &= x_1 \times y_0 \\
z_{11} &= x_1 \times y_1
\end{aligned}
$$

4. Combine the results:

$$
x \times y = 10^n z_{11} + 10^{n/2}(z_{10} + z_{01}) + z_{00}
$$

Correctness

The algorithm maintains these properties:

- **Invariant**: The product follows from the identity:

$$(10^m x_1 + x_0)(10^m y_1 + y_0) = 10^{2m} x_1 y_1 + 10^m (x_1 y_0 + x_0 y_1) + x_0 y_0$$

- **Progress**: Each recursive call works on numbers with half the digits
- **Termination**: Recursion stops when numbers are single-digit

Runtime Analysis

The time complexity can be analyzed using the recurrence relation:

- **Recurrence**: $T(n) = 4T(n/2) + O(n)$
  - $4T(n/2)$ for the 4 recursive multiplications
  - $O(n)$ for additions and digit shifts
-
  - Work doubles each level ($O(n) \to 2O(n) \to 4O(n) \dots$)
  - Bottom dominates with n^2 leaves
- **Total**: $O(n^2)$ (like brute force)

The algorithm therefore runs in $\boxed{O(n^2)}$ time, matching the complexity of grade-school multiplication.

Limitations

This approach provides no asymptotic improvement over the brute-force method because:

- It still requires four multiplications of half-size numbers
- The recurrence relation $T(n) = 4T(n/2) + O(n)$ solves to quadratic time
- The "algebraic trick" of Karatsuba is needed to reduce the number of recursive calls

The key innovation of Karatsuba's algorithm is reducing the number of recursive multiplications from four to three, which changes the recurrence relation to $T(n) = 3T(n/2) + O(n)$ and yields the improved $O(n^{\log_2 3})$ runtime.

Algorithm 2: Karatsuba Multiplication

Karatsuba's optimized algorithm works as follows:

1. If $n = 1$, return $x \times y$ directly
2. Split each number into two halves:

$$x = 10^{n/2} x_1 + x_0$$
$$y = 10^{n/2} y_1 + y_0$$

3. Compute three products recursively:

$$\begin{aligned} z_0 &= x_0 \times y_0 \\ z_2 &= x_1 \times y_1 \\ z_1 &= (x_1 + x_0) \times (y_1 + y_0) - z_2 - z_0 \end{aligned}$$

4. Combine the results:

$$x \times y = 10^n z_2 + 10^{n/2} z_1 + z_0$$

Correctness

The algorithm maintains these properties:

- **Invariant**: The product follows from:

$$(10^m x_1 + x_0)(10^m y_1 + y_0) = 10^{2m} z_2 + 10^m z_1 + z_0$$

  where $z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0$ captures the cross terms

- **Progress**: Each recursive call works on numbers with half the digits
- **Termination**: Recursion stops when numbers are single-digit

Runtime Analysis

The time complexity improves due to fewer recursive calls:

- **Recurrence**: $T(n) = 3T(n/2) + O(n)$
  - $3T(n/2)$ for the 3 recursive multiplications
  - $O(n)$ for additions/subtractions
-
  - Work grows by factor of 1.5 each level ($O(n) \to 1.5O(n) \to 2.25O(n) \dots$)
  - Total work dominated by $O(n^{\log_2 3})$ leaves
- **Total**: $O(n^{\log_2 3}) \approx O(n^{1.585})$

The algorithm therefore runs in $\boxed{O(n^{\log_2 3})}$ time.

Advantages

This approach provides asymptotic improvement because:

- Reduces recursive multiplications from four to three
- The recurrence relation $T(n) = 3T(n/2) + O(n)$ solves to subquadratic time
- The algebraic trick combines additions to eliminate one multiplication

3. Matrix Multiplication

Problem Specification

Given two $n \times n$ matrices A and B, where n is a power of 2, compute their product $C = A \times B$.

Algorithm 1

The standard recursive multiplication works as follows:

1. If $n = 1$, return $A[1,1] \times B[1,1]$
2. Partition each matrix into four submatrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

3. Compute eight products recursively:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

4. Combine the results:

$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Correctness

The algorithm maintains these properties:

- **Invariant**: The product follows from block matrix multiplication rules
- **Progress**: Each recursive call works on matrices half the size
- **Termination**: Recursion stops when matrices are 1×1

Runtime Analysis

The time complexity can be analyzed using the recurrence relation:

- **Recurrence**: $T(n) = 8T(n/2) + O(n^2)$
  - $8T(n/2)$ for the 8 recursive multiplications
  - $O(n^2)$ for matrix additions

- – Work grows by factor of 8 each level ($O(n^2) \to 8O(n^2/4) \to 64O(n^2/16)\dots$)
  – Bottom dominates with $n^{\log_2 8} = n^3$ leaves
- **Total**: $O(n^3)$ (standard cubic time)

The algorithm therefore runs in $\boxed{O(n^3)}$ time.

Limitations

This approach provides no asymptotic improvement because:

- It still requires eight multiplications of half-size matrices
- The recurrence relation $T(n) = 8T(n/2) + O(n^2)$ solves to cubic time
- Strassen's method is needed to reduce the number of recursive calls

Algorithm 2: Strassen's Method

Strassen's optimized algorithm works as follows:

1. If $n = 1$, return $A[1,1] \times B[1,1]$
2. Partition matrices as before
3. Compute seven products recursively:

$$\begin{aligned}
P_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
P_2 &= (A_{21} + A_{22})B_{11} \\
P_3 &= A_{11}(B_{12} - B_{22}) \\
P_4 &= A_{22}(B_{21} - B_{11}) \\
P_5 &= (A_{11} + A_{12})B_{22} \\
P_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
P_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}$$

4. Combine the results:

$$\begin{aligned}
C_{11} &= P_1 + P_4 - P_5 + P_7 \\
C_{12} &= P_3 + P_5 \\
C_{21} &= P_2 + P_4 \\
C_{22} &= P_1 - P_2 + P_3 + P_6
\end{aligned}$$

Correctness

The algorithm maintains these properties:

- **Invariant**: The product can be expressed through these 7 carefully constructed products
- **Progress**: Each recursive call works on matrices half the size
- **Termination**: Recursion stops when matrices are 1×1

Runtime Analysis

The time complexity improves due to fewer recursive calls:

- **Recurrence**: $T(n) = 7T(n/2) + O(n^2)$
  - $7T(n/2)$ for the 7 recursive multiplications
  - $O(n^2)$ for matrix additions/subtractions
-
  - Work grows by factor of 7 each level ($O(n^2) \to 7O(n^2/4) \to 49O(n^2/16)\ldots$)
  - Total work dominated by $O(n^{\log_2 7})$ leaves
- **Total**: $O(n^{\log_2 7}) \approx O(n^{2.807})$

The algorithm therefore runs in $\boxed{O(n^{\log_2 7})}$ time.

Advantages

This approach provides asymptotic improvement because:

- Reduces recursive multiplications from eight to seven
- The recurrence relation $T(n) = 7T(n/2) + O(n^2)$ solves to subcubic time
- The clever combination of additions eliminates one multiplication

4. Maximum Element Problem

Problem Specification

Given an array $A[1..n]$ of numbers, find the maximum element.

Algorithm 1: Basic Divide-and-Conquer (O(n))

1. If $n = 1$, return $A[1]$
2. Divide the array into left and right halves:

$$\text{mid} = \lfloor n/2 \rfloor$$

3. Recursively find:
   - maxLeft in $A[1..\text{mid}]$
   - maxRight in $A[\text{mid} + 1..n]$
4. Return $\max(\text{maxLeft}, \text{maxRight})$

Runtime Analysis

- **Recurrence**: $T(n) = 2T(n/2) + O(1)$
- **Work per level**: Constant time comparison
- **Total work**: $n - 1$ comparisons (same as iterative)
- **Total**: $\boxed{O(n)}$

Algorithm 2: Dynamic Programming Approach

Definition

Let $\max[i]$ be the maximum value in $A[1..i]$.

DP Formulation

$$\max[i] = \begin{cases} A[1] & \text{if } i = 1 \\ \max(\max[i-1], A[i]) & \text{if } i > 1 \end{cases}$$

Implementation

1. Initialize $\max[1] = A[1]$
2. For $i = 2$ to n:
   - $\max[i] \leftarrow \max(\max[i-1], A[i])$
3. Return $\max[n]$

Space Optimization

The DP solution can be optimized to use constant space:

1. Start with currentMax $= A[1]$
2. For each subsequent element $A[i]$ from $i = 2$ to n:
   - Update currentMax $= \max(\text{currentMax}, A[i])$
3. The final value of currentMax is the overall maximum

Why this works:

- At each step, currentMax stores exactly what $\max[i]$ stored in the DP table
- We only need the most recent maximum, not the entire history
- This is the standard iterative maximum-finding algorithm!

Correctness Proof

The DP solution's correctness follows immediately from:

- **Base Case**: When $i = 1$, $\max[1] = A[1]$ is trivially correct
- **Inductive Step**: For $i > 1$, by the recurrence:

$$\max[i] = \max(\underbrace{\max[i-1]}_{\text{Max of } A[1..i-1]}, \underbrace{A[i]}_{\text{New element}})$$

  which correctly maintains the maximum by induction
- **Termination**: After n steps, $\max[n]$ contains the global maximum

$\boxed{\text{The recurrence directly encodes the mathematical definition of maximum}}$

The optimized version is correct because:

- It implements the exact same logic as the DP
- currentMax always equals $\max[i]$ from the full DP table
- Only stores the necessary information (last maximum)

Analysis

- **Time**: $\boxed{O(n)}$ (each element visited once)
- **Space**:
  - Original: $\boxed{O(n)}$ (stores all $\max[i]$)
  - Optimized: $\boxed{O(1)}$ (just tracks current maximum)

5. Maximum Profit Problem

Problem Specification

Given an array $P[1..n]$ of daily stock prices, find the maximum profit obtainable by buying on day i and selling on day $j > i$.

Algorithm 1: Basic Divide-and-Conquer (O(n log n))

1. If $n \leq 1$, return 0 (no transaction possible)
2. Divide the array into left and right halves:

$$\text{mid} = \lfloor n/2 \rfloor$$

3. Recursively find:

- maxProfitLeft in $P[1..\text{mid}]$
- maxProfitRight in $P[\text{mid}+1..n]$

4. Compute maxCrossingProfit:
   - $\text{minPrice} = \min(P[1..\text{mid}])$
   - $\text{maxPrice} = \max(P[\text{mid}+1..n])$
   - $\text{maxCrossingProfit} = \text{maxPrice} - \text{minPrice}$
5. Return $\max(\text{maxProfitLeft}, \text{maxProfitRight}, \text{maxCrossingProfit})$

Runtime Analysis

- **Recurrence**: $T(n) = 2T(n/2) + O(n)$
- **Work per level**: $O(n)$ for min/max computations
- **Total**: $\boxed{O(n \log n)}$

Optimization Insight

The $O(n \log n)$ Diide and Conquer does redundant work:

- Recomputes min/max in subarrays repeatedly
- Each merge costs $O(n)$ for min/max scans

Solution: Return *three values* from each recursion:

- Max profit *and* min/max prices
- Enables $O(1)$ merges using:

$$\begin{aligned} \text{min} &= \min(L_{min}, R_{min}) \\ \text{max} &= \max(L_{max}, R_{max}) \\ \text{profit} &= \max(L_{profit}, R_{profit}, R_{max} - L_{min}) \end{aligned}$$

$$\boxed{\text{Extra return values} \Rightarrow \text{Recurrence improves to } T(n) = 2T(n/2) + O(1) = O(n)}$$

Algorithm 2: Optimized Divde and Conquer (O(n))

Return three values from each recursion:

- maxProfit
- minPrice
- maxPrice

Modified steps:

1. Base case returns $(0, P[1], P[1])$ for $n = 1$

2. Recursively compute tuples $(L_{profit}, L_{min}, L_{max})$ and $(R_{profit}, R_{min}, R_{max})$
3. Compute:

$$\begin{aligned} \text{maxProfit} &= \max(L_{profit}, R_{profit}, R_{max} - L_{min}) \\ \text{minPrice} &= \min(L_{min}, R_{min}) \\ \text{maxPrice} &= \max(L_{max}, R_{max}) \end{aligned}$$

4. Return $(\text{maxProfit}, \text{minPrice}, \text{maxPrice})$

Runtime Analysis

- **Recurrence**: $T(n) = 2T(n/2) + O(1)$
- **Work per level**: Constant time tuple operations
- **Total**: $\boxed{O(n)}$

Algorithm 3: Dynamic Programming (O(n))

Iterative solution tracking MaxProfit seen so far:

Full DP Table Approach

We define two arrays:

- minPrice[i]: the minimum price from day 1 to day i
- maxProfit[i]: the maximum profit achievable by selling on day i

Base Case:

$$\text{minPrice}[1] = P[1], \quad \text{maxProfit}[1] = 0$$

Recurrence: For $i = 2$ to n,

$$\text{minPrice}[i] = \min(\text{minPrice}[i-1], P[i])$$

$$\text{maxProfit}[i] = \max(\text{maxProfit}[i-1], P[i] - \text{minPrice}[i])$$

Final Answer: maxProfit[n]

Space Optimization

Since both states depend only on the previous day, we can reduce space to $O(1)$ by using scalar variables:

1. Initialize:

$$\text{minPrice} \leftarrow P[1], \quad \text{maxProfit} \leftarrow 0$$

2. For $i = 2$ to n:
   - Update minimum price:

$$\text{minPrice} \leftarrow \min(\text{minPrice}, P[i])$$

- Update maximum profit:

$$\text{maxProfit} \leftarrow \max(\text{maxProfit}, P[i] - \text{minPrice})$$

3. Return maxProfit

Runtime Analysis

- Single pass over the input array
- Constant time updates per iteration
- **Time Complexity:** $\boxed{O(n)}$
- **Space Complexity:** $\boxed{O(1)}$

Maximum Subarray Sum Problem

Problem Specification

Given an array $A[1..n]$ of integers, find the maximum sum of any contiguous subarray.

Algorithm 1: Basic Divide-and-Conquer ($O(n \log n)$)

1. **Base Case**: If $n = 1$, return $\max(A[1], 0)$ (assuming empty subarray is allowed with sum 0)
2. **Divide**: Split the array into left and right halves:

$$\text{mid} = \lfloor n/2 \rfloor$$

3. **Conquer**:
   - Recursively find maxLeft in $A[1..\text{mid}]$
   - Recursively find maxRight in $A[\text{mid} + 1..n]$
4. **Combine**:
   - Compute maxCrossing:
     - Find maximum sum ending at mid (left part)
     - Find maximum sum starting at mid + 1 (right part)
     - maxCrossing = leftMax + rightMax
5. Return $\max(\text{maxLeft}, \text{maxRight}, \text{maxCrossing})$

Runtime Analysis

- **Recurrence**: $T(n) = 2T(n/2) + O(n)$
- **Work per level**: $O(n)$ for crossing sum computation
- **Total**: $\boxed{O(n \log n)}$

Optimization Insight

The $O(n \log n)$ Divide and Conquer can be improved:

- Return four values from each recursion:
  - Maximum subarray sum
  - Maximum prefix sum
  - Maximum suffix sum
  - Total sum
- Enables $O(1)$ merge operations

$$\boxed{\text{Extra return values} \Rightarrow \text{Recurrence improves to } T(n) = 2T(n/2) + O(1) = O(n)}$$

Algorithm 2: Optimized Divide-and-Conquer ($O(n)$)

Return four values from each recursion:

- maxSum: Maximum subarray sum
- maxPrefix: Maximum prefix sum
- maxSuffix: Maximum suffix sum
- totalSum: Total sum of the array

Modified steps:

1. **Base case**: For $n = 1$, return $(\max(A[1], 0), \max(A[1], 0), \max(A[1], 0), A[1])$
2. Recursively compute tuples $(L_{max}, L_{prefix}, L_{suffix}, L_{total})$ and $(R_{max}, R_{prefix}, R_{suffix}, R_{total})$
3. Compute:

$$\begin{aligned}
\text{maxSum} &= \max(L_{max}, R_{max}, L_{suffix} + R_{prefix}) \\
\text{maxPrefix} &= \max(L_{prefix}, L_{total} + R_{prefix}) \\
\text{maxSuffix} &= \max(R_{suffix}, R_{total} + L_{suffix}) \\
\text{totalSum} &= L_{total} + R_{total}
\end{aligned}$$

4. Return $(\text{maxSum}, \text{maxPrefix}, \text{maxSuffix}, \text{totalSum})$

Runtime Analysis

- **Recurrence**: $T(n) = 2T(n/2) + O(1)$
- **Work per level**: Constant time tuple operations
- **Total**: $\boxed{O(n)}$

Algorithm 3: Dynamic Programming ($O(n)$)

Full DP Table Approach

We define:

- $\text{dp}[i]$: Maximum subarray sum ending at position i

Base Case:

$$\text{dp}[1] = A[1]$$

Recurrence: For $i = 2$ to n:

$$\text{dp}[i] = \max(A[i], \text{dp}[i-1] + A[i])$$

Final Answer:

$$\max(\text{dp}[1..n])$$

Space Optimization

Since we only need the previous value, we can reduce space to $O(1)$:

1. Initialize:
$$\text{currentMax} = A[1], \quad \text{globalMax} = A[1]$$
2. For $i = 2$ to n:
   - $\text{currentMax} = \max(A[i], \text{currentMax} + A[i])$
   - $\text{globalMax} = \max(\text{globalMax}, \text{currentMax})$
3. Return globalMax

Runtime Analysis

- **Time Complexity**: $\boxed{O(n)}$ (single pass through array)
- **Space Complexity**: $\boxed{O(1)}$ (constant space with optimization)

The 2-Sum Problem and Variations

Problem Specification

Given an array $A[1 \dots n]$ of integers and target S, determine if $\exists$ distinct indices i, j with $A[i] + A[j] = S$.

Solution 1: Naive Brute Force

- Check all possible pairs:
  - For $i = 1$ to $n - 1$
  - For $j = i + 1$ to n
  - Check if $A[i] + A[j] = S$
- Runtime: $\boxed{O(n^2)}$ (all $\binom{n}{2}$ pairs)

Solution 2: Smarter Search with Sorting

First observation: *If we sort the array, we can search more efficiently*

1. Sort the array: $O(n \log n)$
2. For each element $A[i]$:
   - Perform binary search for $(S - A[i])$ in $A[i+1 \ldots n]$
3. Runtime: $\boxed{O(n \log n)}$ (sorting + n binary searches)

But wait! The binary searches are not independent - we're not using information from previous searches. Can we do better?

Solution 3: Two Pointer Technique

Second observation: *We can exploit the sorted order to eliminate many possibilities at once*

1. Sort the array: $O(n \log n)$
2. Initialize $left \leftarrow 1$, $right \leftarrow n$
3. While $left < right$:
   - $sum \leftarrow A[left] + A[right]$
   - If $sum = S$: return `Found`
   - If $sum < S$: increment $left$ (need larger values)
   - Else: decrement $right$ (need smaller values)
4. Runtime: $\boxed{O(n \log n)}$ (sorting dominates)
5. Space: $O(1)$ (in-place)

Solution 4: Optimal Hashing Approach

Final observation: *If we can search in constant time, we can achieve linear runtime*

1. Initialize hash table H (using universal hashing)
2. For each $A[i]$:
   - If $(S - A[i]) \in H$: return `Found`
   - Insert $A[i]$ into H
3. Runtime: $\boxed{O(n)}$ expected (with universal hashing)
4. Space: $O(n)$ (for hash storage)

Progression of Ideas

Approach	Time	Key Insight
Brute Force	$O(n^2)$	Check all possibilities
Sorted + Binary Search	$O(n \log n)$	Reduce search space
Two Pointer	$O(n \log n)$	Exploit sorted order
Hashing	$O(n)$	Constant-time lookups

Note: The hashing approach assumes:

- Good hash function from universal family
- Proper handling of collisions
- $O(1)$ expected time for dictionary operations

3-Sum and 4-Sum Problems

3-Sum Problem Specification

Given an array $A[1 \ldots n]$, find all unique triplets (i, j, k) such that $A[i] + A[j] + A[k] = 0$ with $i < j < k$.

3-Sum Solution: Reduction to 2-Sum

1. Sort the array: $O(n \log n)$
2. For each $A[i]$ from 1 to $n - 2$:
   - Solve 2-Sum for target $-A[i]$ in subarray $A[i + 1 \ldots n]$
   - Use two-pointer technique (since array is sorted)
   - Skip duplicate elements to avoid redundant triplets
3. Runtime: $\boxed{O(n^2)}$
   - $O(n \log n)$ sorting + $O(n)$ per element for 2-Sum
4. Space: $O(1)$ (excluding output storage)

4-Sum Problem Specification

Given an array $A[1 \ldots n]$, find all unique quadruplets (i, j, k, l) with $A[i] + A[j] + A[k] + A[l] = S$ and $i < j < k < l$.

4-Sum Solution: Two-Level Reduction

1. Sort the array: $O(n \log n)$
2. For each $A[i]$ from 1 to $n-3$:
   - For each $A[j]$ from $i+1$ to $n-2$:
   - Solve 2-Sum for target $S - A[i] - A[j]$ in $A[j+1 \ldots n]$
   - Skip duplicates at both outer levels
3. Runtime: $\boxed{O(n^3)}$
   - $O(n^2)$ pairs for outer loops + $O(n)$ per pair for 2-Sum

4-sum with Hashing

1. Precompute all pairs $(A[k], A[l])$ and store their sums in hash map H
2. For each pair $(A[i], A[j])$:
   - Search H for target $S - A[i] - A[j]$
   - Ensure all indices are distinct
3. Runtime: $\boxed{O(n^2)}$ (optimal for 4-Sum)
   - $O(n^2)$ pair generation + $O(1)$ lookups
4. Space: $O(n^2)$ (for storing all pairs)

Median Recursion, Pivots, QuickSort, Selection

Core Principle: Divide-and-Conquer Using Medians

The median is a powerful tool for divide-and-conquer because it guarantees a balanced split into smaller and bigger halves:

- **Perfect Partitioning**: The median divides an array into two equal (or nearly equal) parts.
  - If we could find the exact median in linear time, sorting would follow the optimal recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n).$$

.

Median-Based Sorting

If we can find the median efficiently, we obtain an optimal sorting strategy:

1. Assume we can compute the exact median m in $O(n)$ time (??)
2. **Partitioning**: Split the array into:
   - $A_{\leq m} = \{x \in A \mid x \leq m\}$
   - $A_{>m} = \{x \in A \mid x > m\}$
3. **Recursive Sorting**: Apply the same procedure to both partitions

Runtime Analysis with Exact Median

The perfect balance gives us the best-case recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

where:

- $2T(n/2)$ comes from sorting two equal-sized halves
- $O(n)$ is for median finding and partitioning

This solves to $T(n) = O(n \log n)$.

Approximate Medians Suffice

Interestingly, we don't need perfect balance - any *constant fraction* partition maintains $O(n \log n)$ time:

- Suppose we always partition into sizes αn and $(1-\alpha)n$ for some $0 < \alpha < 1$
- The recurrence becomes:

$$T(n) = T(\alpha n) + T((1-\alpha)n) + O(n)$$

- For any constant α, this still solves to $O(n \log n)$ because:
  - The recursion tree has $O(\log n)$ levels
  - Each level does $O(n)$ work in partitioning
- Example: Even with a 30-70 split ($\alpha = 0.3$), we get:

$$T(n) = T(0.3n) + T(0.7n) + O(n) = O(n \log n)$$

This observation is crucial - it shows that randomized QuickSort works well because:

- Random pivots give reasonably balanced splits on average
- We don't need the exact median, just "good enough" pivots
- The same principle applies to QuickSelect for selection problems

Quicksort

Quicksort follows the divide-and-conquer paradigm with three key steps:

- **Partition**: Rearrange elements around a pivot
- **Conquer**: Recursively sort subarrays
- **Combine**: Trivial (no work needed)

The Partition Algorithm

Given array $A[1:n]$ and pivot index p:

1. Swap $A[p]$ with $A[n]$
2. Initialize $j \leftarrow 1$
3. For $i \leftarrow 1$ to $n-1$:
   - If $A[i] < A[n]$, swap $A[i]$ with $A[j]$ and increment j
4. Swap $A[n]$ with $A[j]$

Runtime Analysis

Partition runs in $\Theta(n)$ time with:

- Exactly $n-1$ comparisons
- At most n swaps

Quicksort Algorithm

1. If $n \leq 1$, return (base case)
2. Choose pivot index p (typically last element)
3. Partition the array around $A[p]$, obtaining q
4. Recursively sort:
   - Left subarray $A[1..q-1]$
   - Right subarray $A[q+1..n]$

Worst-Case Analysis

Occurs when pivot is always minimum/maximum:

$$T(n) = T(n-1) + \Theta(n) \implies T(n) = \sum_{k=1}^{n} \Theta(k) = \Theta(n^2)$$

Best-Case Analysis

With perfect median splits:

$$T(n) = 2T(n/2) + \Theta(n) \implies T(n) = \Theta(n \log n)$$

Randomized QuickSort

Exact median is not easy to compute, so randomization helps:

1. Pick a pivot element uniformly at random
2. Partition the array:

$$A_{\leq p} = \{x \in A \mid x \leq p\}$$
$$A_{>p} = \{x \in A \mid x > p\}$$

3. Recursively sort both partitions

Expected Runtime of Randomized Quicksort

Let $T(A)$ denote the random runtime for sorting an array A of length n. We analyze the expected runtime $\mathbb{E}[T(A)]$.

1. **Pivot Selection:** Choose a pivot index p uniformly at random from $\{1, \ldots, n\}$.
2. **Expected Runtime:**

$$\begin{aligned}
\mathbb{E}[T(A)] &= \sum_{k=1}^{n} \Pr(p = k) \cdot \mathbb{E}[T(A) \mid p = k] \\
&= \frac{1}{n} \sum_{k=1}^{n} \mathbb{E}[T(A[1\,..\,k-1]) + T(A[k+1\,..\,n]) + cn] \\
&= \frac{1}{n} \sum_{k=1}^{n} (\mathbb{E}[T(k-1)] + \mathbb{E}[T(n-k)]) + cn
\end{aligned}$$

3. **Simplifying:**

$$\begin{aligned}
\mathbb{E}[T(n)] &= \frac{1}{n} \sum_{k=1}^{n} (\mathbb{E}[T(k-1)] + \mathbb{E}[T(n-k)]) + cn \\
&= \frac{2}{n} \sum_{i=0}^{n-1} \mathbb{E}[T(i)] + cn
\end{aligned}$$

4. **Solving the Recurrence:** Let $T(n) = \max_{|A|=n} \mathbb{E}[T(A)]$. We can show that

$$T(n) \leq 2cn \ln n$$

   by induction.

QuickSelect to find k-th Smallest Element

Algorithm

1. Partition using random pivot
2. Recurse only on the relevant partition:

$$\text{Recurrence} = \begin{cases} T(|A_{<p}|) + O(n) & \text{if } k \leq |A_{<p}| \\ T(|A_{>p}|) + O(n) & \text{otherwise} \end{cases}$$

Runtime Analysis

1. **Random Pivot is good**: - Chooses element uniformly at random - With probability $\geq \frac{1}{2}$, pivot is in middle 50% of current subarray
2. **Problem Reduction**: - Good pivot reduces problem size to $\leq \frac{3}{4}$ of original - Expected size: $\mathbb{E}[n_{i+1}] \leq \frac{7}{8} n_i$
3. **Total Work**: - $\mathbb{E}[\sum_i n_i] \leq n + \frac{7}{8}n + (\frac{7}{8})^2 n + \cdots$ - Sums to $O(n)$

Deterministic Linear-Time Selection via Median-of-Medians

Use a median of median (computed recursively) as the pivot.

Algorithm

1. **Group & Sort** - Divide into $\lceil n/5 \rceil$ groups of 5 elements - Sort each group: $O(1)$ per group $\Rightarrow O(n)$ total
2. **Find Median of Medians** - Collect all group medians - Recursively find their median M
3. **Partition & Recurse** - Partition array using M as pivot - Recurse on relevant partition based on target rank k

Analysis

- **Approximate Median Guarantee – Balanced Split**:
  - At least half of the group medians are $\leq M$, i.e., $\frac{n}{10}$ medians.
  - Each such median is the 3rd largest in its group, so it is greater than or equal to two other elements in that group.
  - Thus, each of these $\frac{n}{10}$ medians contributes at least 3 elements $\leq M$, totaling at least $\frac{3n}{10}$ elements.
  - Similarly, at least $\frac{3n}{10}$ elements are $\geq M$.
  - Therefore, in the worst case, the recursive call is on at most $\frac{7n}{10}$ elements.
- **Recurrence Relation**:

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

  - $T(n/5)$: Recursion to compute the median of medians.
  - $T(7n/10)$: Worst-case recursive call on the larger partition.
  - $O(n)$: Time for grouping, sorting, and partitioning.

- **Total Runtime**:
  - Each level does $O(n)$ work.
  - The problem size reduces geometrically due to the 30%/70% partition guarantee.
  - Total time: $T(n) = O(n)$.

Median Recursion: Closest Pair Example

Given an array A of n real numbers, find the closest pair of element x, y. that elements with $|x - y$ minimal.

Algorithm: Divide and Conquer with Median

1. **Base Case**:
   - If $|A| \leq 1$: return ∞
   - If $|A| = 2$: return $|A[1] - A[0]|$
2. **Divide**:
   - $m \leftarrow \text{Median}(A)$
   - Partition A into $A_1 = \{x \leq m\}$ and $A_2 = \{x > m\}$
3. **Recurse**:
   - $x_1 \leftarrow \text{ClosestPair}(A_1)$
   - $x_2 \leftarrow \text{ClosestPair}(A_2)$
4. **Combine**:
   - $y \leftarrow (\max A_1 - m) + (m - \min A_2)$
   - Return $\min(x_1, x_2, y)$

Correctness

- **Invariant**: The closest pair is either:
  - Within A_1 (captured by x_1)
  - Within A_2 (captured by x_2)
  - Across the median (captured by $y_1 + y_2$)
- **Termination**: Recursion stops when $|A| \leq 2$

Runtime Analysis

- Recurrence: $T(n) = 2T(n/2) + O(n)$
  - $2T(n/2)$ for recursive calls
  - $O(n)$ for median/min/max operations
- Solves to $O(n \log n)$

Data Structures

Comparison-Based Sorting Lower Bound

- **Decision Tree Model:**
  - Any comparison-based sorting algorithm can be modeled as a binary decision tree.
  - Each internal node represents a comparison between two elements.
  - Each leaf corresponds to one of the $n!$ possible orderings of the input.
- **Lower Bound on Tree Height:**
  - A binary tree with $n!$ leaves must have height at least $\log_2(n!)$.
  - So, the worst-case number of comparisons satisfies:

$$h \geq \log_2(n!) = \Theta(n \log n)$$

- **Implication:**
  - Any comparison-based sorting algorithm must make at least $\Omega(n \log n)$ comparisons in the worst case.

Counting Sort

Problem Specification

Given an array $A[1 \ldots n]$ where each element $A[i] \in \{1, 2, \ldots, M\}$, sort the array in non-decreasing order.

Algorithm

1. Create a counting array $C[1 \ldots M]$ and initialize all entries to 0.
2. For $i = 1$ to n: increment the count $C[A[i]]$ by 1.
3. Let $p \leftarrow 1$. For $j = 1$ to M:
   - While $C[j] > 0$: set $A[p] \leftarrow j$, increment p, and decrement $C[j]$.

Correctness

The correctness of Counting Sort follows from two key properties:

1. **Preservation of Values:** During the counting phase, each input element $A[i]$ increments $C[A[i]]$. Thus, for every $j \in \{1, \ldots, M\}$, the count $C[j]$ stores the exact number of occurrences of j in the input array. The reconstruction phase writes each value j exactly $C[j]$ times, ensuring no values are lost or duplicated.

2. **Sorted Order:** The reconstruction writes values in increasing order of keys $j = 1$ to M. Hence, all smaller values are written before larger ones. Since within each key we write all occurrences consecutively, the final array is sorted in non-decreasing order.

No comparisons are performed: sorting is achieved through counting and indexed placement, relying entirely on the structure of the input domain.

Runtime Analysis

- **Counting phase:** Takes $O(n)$ time to process the input array.
- **Reconstruction phase:** Takes $O(M + n)$ time in the worst case.
- **Total Time Complexity:**

$$T(n, M) = O(n + M)$$

- Counting Sort is optimal when the value range is small, i.e., $M = O(n)$.

Beyond Comparison-Based Sorting

- Comparison-based sorting algorithms (e.g., Merge Sort, Quick Sort, Heap Sort) have a lower bound of $\Omega(n \log n)$ due to the decision tree model.
- Counting Sort breaks this bound by leveraging additional structure: the input elements are integers drawn from a known, bounded domain.

Heaps

Binary Heaps

- Complete binary tree maintaining heap property:
  - **Max-Heap**: $A[parent(i)] \geq A[i]$
  - **Min-Heap**: $A[parent(i)] \leq A[i]$
- Implemented as array with index arithmetic:
  - Parent: $\lfloor (i-1)/2 \rfloor$
  - Left child: $2i + 1$
  - Right child: $2i + 2$

- Operations:
  - `insert(x)`: $O(\log n)$ (bubble-up)
  - `extractMax()/extractMin()`: $O(\log n)$ (heapify-down)
  - `getMax()/getMin()`: $O(1)$

Heap Sort

Problem Specification

Given an array $A[1 \dots n]$, sort the array in non-decreasing order.

Algorithm

1. Build max-heap from unsorted array.
2. Repeatedly extract max and place at end.
3. Final result is sorted in ascending order.

Proof of Correctness

We maintain the following invariants throughout the sorting phase of Heap Sort:

- **Heap Invariant:** At the start of each iteration, the subarray $A[0 \dots i-1]$ satisfies the max-heap property.
- **Sorted Invariant:** The subarray $A[i \dots n-1]$ contains the $n-i$ largest elements of the array in sorted (increasing) order.

At each step, the algorithm swaps the root (maximum element) of the heap with the last element of the heap region, effectively placing the next-largest element in its correct sorted position. The heap size is reduced by one, and the heap property is restored via `heapify`. As a result, the sorted portion of the array grows from right to left, and the heap property is maintained on the remaining elements.

Once the heap size reaches zero, all elements have been placed in sorted order.

Complexity Analysis

- **Build Heap:** $O(n)$
- **Sort Phase:** n removals, each followed by `heapify` taking $O(\log n)$, so total is $O(n \log n)$
- **Space:** $O(1)$ (in-place)

Sorting a k-Sorted Array

Problem Specification

Given an array $A[1 \dots n]$ where each element is at most k positions away from its correct sorted position, sort the array in non-decreasing order.

Algorithm

Use a min-heap of size $k+1$ to maintain the smallest $k+1$ elements that could be in the current position:

1. Create a min-heap and insert the first $k+1$ elements of A.
2. Initialize a write index $p \leftarrow 1$.
3. For $i = k+2$ to n:
   - Extract the minimum from the heap and write it to $A[p]$, increment p.
   - Insert $A[i]$ into the heap.
4. While the heap is not empty:
   - Extract the minimum and write it to $A[p]$, increment p.

Correctness

Since the array is k-sorted, every element is at most k positions away from where it should be in the fully sorted array. That means:

- The smallest element in the entire array must be somewhere in the first $k+1$ elements. So if we build a min-heap from $A[1 \ldots k+1]$, extracting the minimum gives us the first correct element.
- After placing that, the next smallest element must be within the next $k+1$ values, i.e., between $A[2]$ and $A[k+2]$. The heap already has the remaining k elements from the previous window, and we add one new element from the array. So the heap still contains all the right candidates.
- This continues for every position. On step i, we're looking for the i-th smallest element. Because of the k-sorted property, this element must be somewhere between $A[i]$ and $A[i+k]$, and all of those values are in the heap (except the ones we've already removed earlier).
- So, at each step, we're guaranteed that the heap contains the correct next element to place, and `extract-min` always gives us the right one.

By repeating this until the heap is empty, we build the sorted array correctly.

Runtime Analysis

- **Heap Initialization:** Insert $k+1$ elements into the heap: $O(k)$
- **Main loop:** Each of the n elements is inserted and one is removed from the heap.
- **Per operation cost:** Insert and extract-min take $O(\log k)$
- **Total Time Complexity:**

$$T(n,k) = O(n \log k)$$

- **Space:** $O(k)$ for the heap

When to Use

This algorithm is highly efficient when $k \ll n$. In such cases, it significantly outperforms comparison-based sorts like Merge Sort and Heap Sort, which take $O(n \log n)$ time regardless of input structure.

Hashing

Hashing provides a way to map keys from a potentially large universe U to indices in a smaller array (hash table) $T[0 \dots m-1]$. The goal is efficient insertion, deletion, and search operations, ideally in $O(1)$ time on average. Hash maps (or dictionaries) are common implementations of this concept, storing (key, value) pairs.

Hash Tables: The $O(1)$ Average Lookup

Hash tables use a **hash function** $h : U \to \{0, \dots, m-1\}$ to compute an index (or "hash code") for each key.

The Collision Problem

- Since the key universe U is usually much larger than the table size m, multiple keys might map to the same index ($h(x) = h(y)$ for $x \neq y$). This is a **collision**.
- **Pigeonhole Principle:** Collisions are inevitable if $n > m$.
- **Challenge:** If we use a fixed, deterministic hash function, an adversary can choose inputs that all hash to the same location, degrading performance to $O(n)$ for lookups in the worst case.

Collision Resolution: Chaining

A common strategy to handle collisions:

- Each table slot $T[i]$ points to a linked list (or other data structure) containing all keys k such that $h(k) = i$.
- **Insert:** Compute $h(k)$, insert k into the list at $T[h(k)]$.
- **Search:** Compute $h(k)$, search the list at $T[h(k)]$.
- **Delete:** Compute $h(k)$, search and remove k from the list at $T[h(k)]$.

The runtime depends on the length of the lists.

Randomization: Universal Hashing

To prevent worst-case behavior from adversarial inputs, we choose the hash function randomly at runtime from a predefined family $\mathcal{H}$.

- **Universal Hash Family:** A family $\mathcal{H}$ is universal if, for any distinct keys $x \neq y$, the probability (over the random choice of $h \in \mathcal{H}$) that they collide is low:

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$$

  (Other definitions like near-universal exist, e.g., $\Pr[\ldots] \leq \frac{c}{m}$ for some constant c).

- **Benefit:** No matter the input keys, the expected number of collisions involving a specific key x is small.

- **Expected Performance (Chaining with Universal Hashing):**
  - Let $\alpha = n/m$ be the load factor.
  - Expected length of a chain: $O(1 + \alpha)$.
  - Expected time for Search/Insert/Delete: $O(1 + \alpha)$.
  - If we maintain $m = \Theta(n)$ (e.g., by resizing the table), the expected time is $\boxed{O(1)}$.

Applications of Hashing

Hash tables (dictionaries/sets) provide efficient solutions for many problems involving lookups, frequency counts, or grouping.

Problem: Two Sum (Revisited)

Problem Specification: Given an array $A[0 \ldots n-1]$ of numbers and a target sum T, find distinct indices i, j such that $A[i] + A[j] = T$, or determine that no such pair exists.

Algorithm (Hashing):

1. Initialize an empty dictionary `D` (stores number $\rightarrow$ index).
2. Iterate through the array with index `i` from 0 to n-1:
   - Calculate the required complement: `complement = T - A[i]`.
   - Check if `complement` exists as a key in the dictionary `D`.
   - If `complement` is found in `D` and `D[complement] != i` (to ensure distinct indices if needed):
     - Return the indices `D[complement]` and `i`.
   - Add the current element and its index to the dictionary: `D[A[i]] = i`. *(Handle potential overwrites based on problem specifics if duplicate numbers exist).*
3. If the loop finishes without finding a pair, return NIL (or an appropriate indicator).

Runtime Analysis:

- **Time Complexity:** $O(n)$ expected time. Each lookup and insertion in the hash table takes $O(1)$ expected time, and we perform n such operations.
- **Space Complexity:** $O(n)$ to store the dictionary in the worst case (all elements are distinct).

Problem: Remove Duplicates (Preserving Order)

Problem Specification: Given an array $A[0 \dots n-1]$, return a new array B containing the unique elements of A in their original relative order of first appearance.

Algorithm (Hashing):

1. Initialize an empty list `B`.
2. Initialize an empty dictionary (used as a hash set) `D` to keep track of elements already seen.
3. Iterate through the input array `A` with index `i` from 0 to n-1:
   - Check if the current element `A[i]` is present as a key in `D`.
   - If `A[i]` is **not** in `D`:
     - Add `A[i]` to `D` (e.g., `D[A[i]] = True`).
     - Append `A[i]` to the result list `B`.
4. Return the list `B`.

Runtime Analysis:

- **Time Complexity:** $O(n)$ expected time. Each lookup and insertion in the hash set takes $O(1)$ expected time.
- **Space Complexity:** $O(k)$ where k is the number of unique elements in A. This is $O(n)$ in the worst case for both the hash set `D` and the result list `B`.

Problem: Equal Sums

Problem Specification: Given an array $A[0 \dots n-1]$ containing unique elements, find if there exist four distinct indices i, j, p, q such that $A[i] + A[j] = A[p] + A[q]$. If such indices exist, return one set; otherwise, indicate failure.

Algorithm (Hashing):

1. Initialize an empty dictionary `D` (stores sum $\rightarrow$ pair of indices `(i, j)` that produced the sum).
2. Iterate with index `i` from 0 to n-2:
   - Iterate with index `j` from `i+1` to n-1:
     - Compute the sum: `current_sum = A[i] + A[j]`.
     - Check if `current_sum` exists as a key in `D`.
     - If `current_sum` is found in `D`:
       * Retrieve the stored pair of indices: `(p, q) = D[current_sum]`.
       * Check if the indices i, j, p, q are all distinct. If they are, return `(i, j, p, q)`.
     - If `current_sum` is **not** found in `D`:
       * Add the sum and the current pair of indices to the dictionary: `D[current_sum] = (i, j)`.

3. If the loops complete without finding such distinct indices, return NIL.

Runtime Analysis:

- **Time Complexity:** $O(n^2)$. The dominant cost is the nested loops iterating through all $\binom{n}{2}$ pairs. Hash table lookups and insertions take $O(1)$ expected time per pair.
- **Space Complexity:** $O(n^2)$ in the worst case, as the dictionary might need to store up to $\binom{n}{2}$ distinct sums and their corresponding index pairs.

Problem: Equal Arrays

Problem Specification: Given two arrays A and B, determine if they contain the same elements with the same frequencies (i.e., if B is a permutation of A).

Algorithm (Hashing):

1. Check if the lengths of A and B are equal. If not, return `False`. Let n be the length.
2. Initialize an empty dictionary `freq_A` to store frequency counts for elements in A.
3. Iterate through array A: For each element `x`, increment its count in `freq_A`. (If `x` is not in `freq_A`, initialize its count to 1).
4. Initialize an empty dictionary `freq_B`.
5. Iterate through array B: For each element `y`, increment its count in `freq_B`.
6. Compare the two frequency dictionaries `freq_A` and `freq_B`. Return `True` if they are identical (contain the same keys with the same corresponding values), and `False` otherwise.

Runtime Analysis:

- **Time Complexity:** $O(n)$ expected time, assuming n is the length of the arrays. Building each frequency map takes $O(n)$ expected time. Comparing the dictionaries takes time proportional to the number of unique elements, which is at most $O(n)$.
- **Space Complexity:** $O(k)$ where k is the number of unique elements. This is $O(n)$ in the worst case to store the frequency maps.

Randomization in Algorithms: The Find-1 Example

Sometimes, introducing randomness into an algorithm can provide better performance guarantees (in expectation or with high probability) compared to deterministic approaches, especially when facing potentially adversarial inputs.

Problem: Find-1

Problem Specification: Given an array $A[1 \dots n]$ containing only 0s and 1s, where it's guaranteed that at least 25% of the elements are 1s. The goal is to find and return the index i of any element such that $A[i] = 1$.

Deterministic Approach

Algorithm:

1. Iterate through the array from index $i = 1$ to n.
2. If $A[i] = 1$, return i.

Runtime Analysis:

- **Best Case:** $O(1)$ (if $A[1] = 1$).
- **Worst Case:** $\Theta(n)$. If all the 1s are clustered at the end of the array (e.g., the first $0.75n$ elements are 0), the algorithm must scan most of the array.
- **Average Case (Random Input):** Often fast, but a deterministic algorithm can be vulnerable to specifically crafted "bad" inputs.

This deterministic approach is like a **deterministic heuristic**: often fast, but potentially slow on some inputs.

Randomized Approach (Las Vegas)

Algorithm:

1. Repeat indefinitely:
   - Choose an index i uniformly at random from $\{1, 2, \ldots, n\}$.
   - If $A[i] = 1$, return i.

Runtime Analysis:

- **Correctness:** The algorithm always returns a correct index (it only returns when it finds a 1). This makes it a **Las Vegas** algorithm (always correct, runtime is random).
- **Expected Runtime:** Let p be the probability of success in one iteration. Since at least 25% of elements are 1, $p \geq 0.25$. The expected number of trials until the first success in a sequence of Bernoulli trials with success probability p is $1/p$. Therefore, the expected number of iterations is $1/p \leq 1/0.25 = 4$. Since each iteration takes $O(1)$ time (random index selection and array lookup), the expected runtime is $\boxed{O(1)}$.
- **Worst-Case Runtime:** Theoretically unbounded. It's possible (though increasingly improbable) to keep picking indices corresponding to 0s forever.

Why Randomization Helps (Connecting to Hashing)

- **Avoiding Worst Cases:** Just like the deterministic Find-1 algorithm has a bad input (1s at the end), deterministic hash functions have bad input sets (keys that collide frequently).

- **Robustness:** A randomized algorithm (like randomized Find-1 or hashing with a universal family) performs well *in expectation* regardless of the specific input distribution. The randomness comes from the algorithm's choices (which index to pick, which hash function to use), not from assumptions about the input.
- **Guarantees:** While deterministic heuristics might fail badly on certain inputs, randomized algorithms often provide strong probabilistic guarantees (e.g., $O(1)$ expected time for hashing operations, $O(n \log n)$ expected time for Randomized Quicksort). They are much harder for an adversary to consistently defeat.
- **Las Vegas vs. Monte Carlo:**
  - **Las Vegas:** Always gives the correct answer; runtime varies (e.g., Randomized Quicksort, Randomized Find-1).
  - **Monte Carlo:** Runtime is fixed (usually fast); the answer might be incorrect with a small probability (e.g., primality testing algorithms like Miller-Rabin).

Using universal hashing for hash maps is an example of applying randomization (Las Vegas style for expected runtime) to achieve robust $O(1)$ expected performance for dictionary operations, avoiding the $O(n)$ worst-case pitfalls of fixed hash functions.

Dynamic Programming

Core Idea: Dynamic Programming (DP) is an algorithmic technique for solving complex problems by breaking them down into simpler, **overlapping subproblems**. It solves each subproblem just once and stores its solution, typically in an array or hash table (memoization table). When the same subproblem is encountered again, the stored solution is retrieved instead of recomputing it. This avoids redundant work, leading to significant speedups for problems that exhibit optimal substructure and overlapping subproblems.

Key Characteristics for DP Applicability:

- **Optimal Substructure:** An optimal solution to the overall problem can be constructed efficiently from optimal solutions to its subproblems.
- **Overlapping Subproblems:** A recursive solution to the problem involves solving the same subproblems multiple times. DP exploits this by computing each subproblem only once.

General Approaches

There are two primary ways to implement a dynamic programming solution:

- **Memoization (Top-Down):**
  - Write a standard recursive function to solve the problem.
  - Before making a recursive call for a subproblem, check if its solution is already stored in a memoization table (e.g., an array or hash map initialized with a special "undefined" value).

  - If the result is found, return it directly.
  - Otherwise, compute the result recursively, store it in the table, and then return it.
  - **Pros:** Often mimics the recursive structure directly, easier to code initially, only computes subproblems that are actually needed.
  - **Cons:** Can have overhead due to recursion depth, runtime analysis might seem less straightforward initially.

- **Tabulation (Bottom-Up):**
  - Determine an order to compute the solutions to subproblems such that whenever the solution to a subproblem is needed, it has already been computed.
  - Typically involves iteratively filling a table (e.g., a 1D or 2D array) starting from the base cases (smallest subproblems) and building up towards the final solution.
  - **Pros:** Avoids recursion overhead, runtime analysis is often clearer (based on loops iterating over the table dimensions).
  - **Cons:** Requires careful thought about the correct computation order, might compute subproblems that aren't strictly necessary for the final answer (though usually doesn't affect asymptotic complexity).

Both memoization and tabulation generally yield the same asymptotic time complexity for a given DP formulation.

Steps for Designing DP Algorithms

A systematic approach can help:

1. **Identify Subproblems:** Define precisely what subproblem needs to be solved. Often involves prefixes, suffixes, ranges, or subsets related to the input parameters (e.g., first i items, target value j).

2. **Recursive Specification:** Define a function (e.g., Solve(i, j)) that represents the optimal solution to the identified subproblem. Clearly state what this function computes.

3. **Recursive Relation:** Derive a recurrence relation that expresses the solution to a larger subproblem in terms of solutions to smaller, overlapping subproblems. Consider the choices available at each step and how they lead to optimal substructure.

4. **Base Cases:** Identify the smallest subproblems whose solutions are known directly without recursion.

5. **Proof of Correctness:** Argue why the recurrence relation correctly captures the optimal solution based on the choices made and the optimal solutions of the subproblems it depends on. Often involves case analysis or induction.

6. **Implementation (Memoization or Tabulation):** Choose an implementation strategy and translate the recurrence into code.

7. **Complexity Analysis:** Determine the number of distinct subproblems and the time taken to compute each one (usually constant time after the recursive calls/table lookups). Total time is typically (Number of Subproblems)×(Time per Subproblem). Analyze space complexity based on the table size.

Example 1: Fibonacci Numbers

A classic illustration of overlapping subproblems.

Problem Specification

Compute the n-th Fibonacci number F_n, where the sequence is defined as:

$$F_n = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = 2 \\ F_{n-1} + F_{n-2} & \text{if } n > 2 \end{cases}$$

(Note: Sometimes defined with $F_0 = 0, F_1 = 1$. We use the $F_1 = 1, F_2 = 1$ definition from the slides).

Recursive Formulation

Specification: Compute F_n. **Recurrence Relation:** Directly given by the definition above. **Base Cases:** $F_1 = 1, F_2 = 1$.

Naive Recursive Algorithm (`RecFibo(n)`):

```
RecFibo(n):
  if n == 1 or n == 2:
    return 1
  else:
    return RecFibo(n-1) + RecFibo(n-2)
```

Analysis: The runtime recurrence is $T(n) = T(n-1) + T(n-2) + O(1)$. This grows exponentially, $T(n) = \Theta(\phi^n) \approx \Theta(1.618^n)$, because the same subproblems (e.g., F_k for $k < n$) are recomputed many times in different branches of the recursion tree.

Proof of Correctness

The recursive algorithm directly implements the mathematical definition, so it's correct by definition.

Memoization Approach (`MemFibo(n)`)

Algorithm:

1. Initialize a global or shared array $F[1..n]$ with a special value (e.g., -1 or 'undefined').
2. **MemFibo(k):**
   - If $F[k]$ is not 'undefined', return $F[k]$.
   - If $k = 1$ or $k = 2$, set result = 1.
   - Else, compute result = MemFibo$(k-1)$ + MemFibo$(k-2)$.

   - Store the result: $F[k] = \text{result}$.
   - Return result.

3. Initial call: `MemFibo(n)`.

Runtime Analysis:

- Each subproblem F_k (for $k = 1 \ldots n$) is computed only once. The first time it's called, it performs the computation; subsequent calls return the stored value in $O(1)$ time.
- The computation inside each call (excluding recursive calls) takes $O(1)$ time (check table, base case check, addition).
- **Time Complexity:** $O(n)$.
- **Space Complexity:** $O(n)$ for the memoization table F and recursion stack depth.

Tabulation Approach (`DynamicFibo(n)`)

Algorithm:

1. Create an array $F[1..n]$.
2. Set $F[1] = 1$.
3. If $n \geq 2$, set $F[2] = 1$.
4. Iterate i from 3 to n:
   - Compute $F[i] = F[i-1] + F[i-2]$.
5. Return $F[n]$.

Runtime Analysis:

- The loop runs $n - 2$ times. Each iteration takes $O(1)$ time.
- **Time Complexity:** $O(n)$.
- **Space Complexity:** $O(n)$ for the table F.

Space Optimization

Notice that computing $F[i]$ only requires $F[i-1]$ and $F[i-2]$. We don't need the entire table. We can compute F_n using only constant extra space. **Algorithm (Space Optimized):**

1. If $n \leq 2$, return 1.
2. Initialize $a = 1$ (F_{n-2}), $b = 1$ (F_{n-1}).
3. Iterate i from 3 to n:
   - Compute current $= a + b$.

- Update $a = b$.
- Update $b =$ current.

4. Return b.

Analysis: Time $O(n)$, Space $O(1)$.

Example 2: 0/1 Knapsack Problem

Problem Specification

Given n items, where item i has weight w_i and value v_i, and a knapsack with maximum weight capacity W. Find a subset S of items such that the total weight $\sum_{i \in S} w_i \leq W$ and the total value $\sum_{i \in S} v_i$ is maximized.

Recursive Formulation

Specification: Let $K(i, j)$ be the maximum value achievable using a subset of items from $\{1, 2, \ldots, i\}$ with a maximum weight capacity of j. The goal is to compute $K(n, W)$.

Recurrence Relation: Consider item i. There are two choices:

1. **Exclude item i:** The maximum value is obtained using items $\{1, \ldots, i-1\}$ with capacity j. Value $= K(i-1, j)$.

2. **Include item i:** This is only possible if $w_i \leq j$. The value is v_i plus the maximum value obtained using items $\{1, \ldots, i-1\}$ with the remaining capacity $j - w_i$. Value $= v_i + K(i-1, j-w_i)$.

We take the maximum of the valid choices:

$$K(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ K(i-1,j) & \text{if } w_i > j \\ \max\{K(i-1,j), v_i + K(i-1, j-w_i)\} & \text{if } w_i \leq j \end{cases}$$

Base Cases: $K(0, j) = 0$ for all $j \geq 0$ (no items, no value). $K(i, 0) = 0$ for all $i \geq 0$ (no capacity, no value).

Proof of Correctness

The proof follows by induction on i. **Base Case ($i = 0$):** $K(0, j) = 0$ is correct as no items yield zero value. **Inductive Step:** Assume $K(i-1, k)$ is correctly computed for all $k \leq W$. To compute $K(i, j)$, we consider the optimal solution for items $\{1, \ldots, i\}$ with capacity j.

- If item i is not included in this optimal solution, the value must be the optimal value using items $\{1, \ldots, i-1\}$ with capacity j, which is $K(i-1, j)$ by the inductive hypothesis.

- If item i is included (only possible if $w_i \leq j$), then the remaining items must form an optimal solution for items $\{1, \ldots, i-1\}$ with capacity $j - w_i$. The total value is $v_i + K(i-1, j-w_i)$, using the inductive hypothesis again.

The recurrence correctly chooses the maximum of these possibilities (or only the first if $w_i > j$), thus $K(i, j)$ is computed correctly.

Memoization Approach (`MemKnapsack(i, j)`)

Algorithm:

1. Initialize a 2D array $D[0..n][0..W]$ with an 'undefined' value.
2. **MemKnapsack(i, j):**
   - If $i = 0$ or $j = 0$, return 0.
   - If $D[i][j] \neq$ undefined, return $D[i][j]$.
   - If $w_i > j$, compute result = MemKnapsack$(i-1, j)$.
   - Else, compute val_exclude = MemKnapsack$(i-1, j)$.
   - Compute val_include = $v_i +$ MemKnapsack$(i-1, j-w_i)$.
   - Set result = max(val_exclude, val_include).
   - Store $D[i][j] =$ result.
   - Return result.
3. Initial call: `MemKnapsack(n, W)`.

Runtime Analysis:

- Number of distinct subproblems (i, j) is $(n+1)(W+1) = O(nW)$.
- Each subproblem is computed once. The computation takes $O(1)$ time excluding recursive calls.
- **Time Complexity:** $O(nW)$.
- **Space Complexity:** $O(nW)$ for the memoization table and recursion stack.

Tabulation Approach (`DynamicKnapsack`)

Algorithm:

1. Create a 2D array $D[0..n][0..W]$.
2. Initialize the first row and first column to 0: $D[0][j] = 0$ for $j = 0..W$, $D[i][0] = 0$ for $i = 0..n$.
3. Iterate i from 1 to n:
   - Iterate j from 1 to W:
     - If $w_i > j$: set $D[i][j] = D[i-1][j]$.
     - Else: set $D[i][j] = \max\{D[i-1][j], v_i + D[i-1][j-w_i]\}$.
4. Return $D[n][W]$.

Runtime Analysis:

- Two nested loops iterate over the $O(nW)$ table entries. Each entry calculation takes $O(1)$.
- **Time Complexity:** $O(nW)$.
- **Space Complexity:** $O(nW)$ for the table D.

Space Optimization

To compute row i, we only need values from row $i-1$. We can optimize space by using only two rows or even just one row (if iterating j carefully). **Algorithm (1D Array):**

1. Create a 1D array $D[0..W]$, initialized to 0.
2. Iterate i from 1 to n:
   - Iterate j from W down to w_i: *(Crucial: Iterate backwards to use values from the 'previous' iteration)*
     - $D[j] = \max(D[j], v_i + D[j - w_i])$.
3. Return $D[W]$.

Analysis: Time $O(nW)$, Space $O(W)$.

Example 3: Minimum Bills (Coin Change)

Problem Specification

Given a target amount N and a set of available bill denominations $\{d_1, d_2, \ldots, d_k\}$ (e.g., $\{1, 5, 14, 22\}$), find the minimum number of bills required to make the amount N. Assume an infinite supply of each denomination.

Recursive Formulation

Specification: Let $T(i)$ be the minimum number of bills required to make amount i. Goal: Compute $T(N)$.

Recurrence Relation: To make amount i, we must have used some last bill d_m (where $d_m \leq i$). The remaining amount $i - d_m$ must have been formed using the minimum number of bills, which is $T(i - d_m)$. We want the choice of d_m that minimizes the total count.

$$T(i) = 1 + \min\{T(i - d_m) \mid d_m \in \{1, 5, 14, 22\} \text{ and } d_m \leq i\}$$

Base Case: $T(0) = 0$ (zero bills needed for amount 0).

Proof of Correctness

Base Case ($i = 0$): $T(0) = 0$ is correct. **Inductive Step:** Assume $T(k)$ is correctly computed for all $k < i$. Any optimal solution for amount i must use at least one bill, say d_m. The remaining $i - d_m$ amount must also be formed optimally (otherwise, we could substitute a better solution for $i - d_m$ and improve the solution for i, contradiction). By the inductive hypothesis, the optimal solution for $i - d_m$ takes $T(i - d_m)$ bills. Thus, using bill d_m leads to a total of $1 + T(i - d_m)$ bills. The recurrence considers all possible last bills d_m and chooses the one minimizing this total count, thus ensuring $T(i)$ is the minimum number of bills for amount i.

Tabulation Approach

Algorithm:

1. Let denominations be $D = \{d_1, \ldots, d_k\}$.
2. Create an array $T[0..N]$.
3. Initialize $T[0] = 0$.
4. Initialize $T[i] = \infty$ for $i = 1$ to N.
5. Iterate i from 1 to N:
   - Iterate through each denomination $d_m \in D$:
     - If $d_m \leq i$:
       - $T[i] = \min(T[i], 1 + T[i - d_m])$.
6. Return $T[N]$.

(Memoization is also straightforward).

Reconstructing the Solution: To find the actual bills used, we can store which denomination led to the minimum at each step.

1. Create an auxiliary array $S[0..N]$.
2. When updating $T[i] = \min(T[i], 1 + T[i - d_m])$, if the minimum came from using bill d_m, store $S[i] = d_m$.
3. To get the bills for N: Start at current $= N$. While current > 0: Add bill $S[\text{current}]$ to the list, update current $=$ current $- S[\text{current}]$.

Runtime Analysis:

- Let $k = |D|$ be the number of denominations.
- The outer loop runs N times. The inner loop runs k times.
- **Time Complexity:** $O(N \cdot k)$.
- **Space Complexity:** $O(N)$ for the table T (and S if reconstructing).

Example 4: Longest Increasing Subsequence (LIS)

Problem Specification

Given an array $A[1..n]$ of distinct integers, find the length of the longest subsequence B of A such that the elements in B are in strictly increasing order.

Recursive Formulation ($O(n^2)$ Approach)

Specification: Let $LIS(i)$ be the length of the longest increasing subsequence of $A[1..i]$ that *ends at* $A[i]$. The final answer is $\max_{1 \le i \le n} LIS(i)$.

Recurrence Relation: To find the LIS ending at $A[i]$, we consider all $j < i$. If $A[j] < A[i]$, then we can potentially append $A[i]$ to the LIS ending at $A[j]$. We want the longest such subsequence.

$$LIS(i) = 1 + \max(\{LIS(j) \mid 1 \le j < i \text{ and } A[j] < A[i]\} \cup \{0\})$$

The max over the empty set is taken as 0 (or $-\infty$ if adding 1), representing the case where $A[i]$ starts a new LIS of length 1. **Base Case:** Implicitly, $LIS(1) = 1$.

Proof of Correctness

Base Case ($i = 1$): The LIS ending at $A[1]$ is just $A[1]$, length 1. $LIS(1) = 1 + \max(\emptyset \cup \{0\}) = 1$, which is correct. **Inductive Step:** Assume $LIS(j)$ is correctly computed for all $j < i$. An LIS ending at $A[i]$ must have $A[i]$ as its last element. The second-to-last element must be some $A[j]$ where $j < i$ and $A[j] < A[i]$. To maximize the length ending at $A[i]$, the subsequence ending at $A[j]$ must itself be an LIS ending at $A[j]$. By the inductive hypothesis, its length is $LIS(j)$. Therefore, the length ending at $A[i]$ by choosing $A[j]$ as the previous element is $1 + LIS(j)$. The recurrence correctly takes the maximum over all valid preceding elements $A[j]$, or sets the length to 1 if no such j exists.

Memoization Approach (`MemLIS(i)`)

Algorithm:

1. Initialize array $T[1..n]$ to 'undefined'.
2. **MemLIS(i):**
   - If $T[i] \neq$ undefined, return $T[i]$.
   - Initialize max_prev_lis = 0.
   - For $j = 1$ to $i - 1$:
     - If $A[j] < A[i]$:
       - max_prev_lis = $\max(\text{max_prev_lis}, \text{MemLIS}(j))$.
   - Set $T[i] = 1 + \text{max_prev_lis}$.
   - Return $T[i]$.
3. Compute MemLIS(k) for all $k = 1..n$.
4. Return $\max_{1 \le k \le n} T[k]$.

Runtime Analysis:

- There are n distinct subproblems $LIS(i)$.
- Computing $LIS(i)$ involves a loop iterating up to $i - 1$ times, with each step taking $O(1)$ time (after memoized recursive calls return).

- Total time: $\sum_{i=1}^{n} O(i) = O(n^2)$.
- **Time Complexity:** $O(n^2)$.
- **Space Complexity:** $O(n)$ for the table and recursion stack.

Tabulation Approach (`DynamicLIS`)

Algorithm:

1. Create an array $T[1..n]$.
2. Iterate i from 1 to n:
   - Initialize $T[i] = 1$. *(Default length if no smaller preceding element)*
   - Iterate j from 1 to $i - 1$:
     - If $A[j] < A[i]$:
       - $T[i] = \max(T[i], 1 + T[j])$.
3. Find the maximum value in the array T: result $= \max_{1 \leq k \leq n} T[k]$.
4. Return result.

Runtime Analysis:

- Two nested loops give $O(n^2)$ iterations. Inner work is $O(1)$. Final max takes $O(n)$.
- **Time Complexity:** $O(n^2)$.
- **Space Complexity:** $O(n)$ for the table T.

Note: A more optimized $O(n \log n)$ DP solution exists for LIS using patience sorting concepts or binary search on auxiliary data structures.

Example 5: Subset Sum Problem

Problem Specification

Given a set $S = \{s_0, s_1, \ldots, s_{n-1}\}$ of non-negative integers and a target integer B, determine if there exists a subset $S' \subseteq S$ such that the sum of elements in S' is exactly B.

Recursive Formulation

Specification: Let $T(i, j)$ be a boolean value indicating whether a subset of $\{s_0, \ldots, s_i\}$ can sum to j. Goal: Compute $T(n - 1, B)$.

Recurrence Relation: Consider element s_i. A sum j can be achieved using elements $\{s_0, \ldots, s_i\}$ if:

1. The sum j can be achieved using only elements $\{s_0, \ldots, s_{i-1}\}$ (i.e., without using s_i). This corresponds to $T(i - 1, j)$.

2. OR, the element s_i is used, and the remaining sum $j - s_i$ can be achieved using elements $\{s_0, \ldots, s_{i-1}\}$. This requires $j \geq s_i$ and corresponds to $T(i-1, j-s_i)$.

$$T(i,j) = T(i-1,j) \vee (j \geq s_i \wedge T(i-1, j-s_i))$$

Base Cases:

- $T(i, 0) =$ TRUE for all $i \geq 0$ (the empty set sums to 0).
- $T(-1, j) =$ FALSE for $j > 0$ (no elements cannot make a positive sum).
- Equivalently, initialize $T(0, j)$ based only on s_0: $T(0, j) =$ TRUE if $j = 0$ or $j = s_0$, else FALSE.

Proof of Correctness

Base Cases: Handled correctly by initialization (empty set sums to 0). **Inductive Step:** Assume $T(i-1, k)$ is correct for all $k \leq B$. We want to show $T(i, j)$ is correct.

- If there is a subset of $\{s_0, \ldots, s_i\}$ summing to j:
  - If this subset does not include s_i, it's a subset of $\{s_0, \ldots, s_{i-1}\}$ summing to j, so $T(i-1, j)$ must be TRUE by hypothesis.
  - If this subset includes s_i, the remaining elements form a subset of $\{s_0, \ldots, s_{i-1}\}$ summing to $j - s_i$, so $T(i-1, j-s_i)$ must be TRUE by hypothesis.

  In either case, the recurrence evaluates to TRUE.
- If the recurrence evaluates to TRUE:
  - If $T(i-1, j)$ is TRUE, then a subset exists using only $\{s_0, \ldots, s_{i-1}\}$, which is also a subset of $\{s_0, \ldots, s_i\}$.
  - If $T(i-1, j-s_i)$ is TRUE (and $j \geq s_i$), then there's a subset of $\{s_0, \ldots, s_{i-1}\}$ summing to $j - s_i$. Adding s_i to this subset yields a subset of $\{s_0, \ldots, s_i\}$ summing to j.

  In either case, a valid subset exists.

The recurrence correctly determines the existence of the subset.

Tabulation Approach

Algorithm:

1. Create a 2D boolean table $T[0..n-1][0..B]$. (Can use indices $0..n$ for items if preferred, adjusting base cases).
2. Initialize base cases: $T[i][0] =$ TRUE for all i. Handle the first item s_0: Set $T[0][s_0] =$ TRUE (if $s_0 \leq B$), all other $T[0][j] =$ FALSE for $j > 0$.
3. Iterate i from 1 to $n-1$ (for items s_i):
   - Iterate j from 1 to B (for target sums):

  - Set $T[i][j] = T[i-1][j]$. *(Case: don't use item s_i)*
  - If $j \geq s_i$:
    * $T[i][j] = T[i][j] \vee T[i-1][j-s_i]$. *(Case: use item s_i)*

4. Return $T[n-1][B]$.

Runtime Analysis:

- The table size is $n \times (B+1)$.
- Filling each entry takes $O(1)$ time.
- **Time Complexity:** $O(nB)$. This is pseudo-polynomial time.
- **Space Complexity:** $O(nB)$. Can be optimized to $O(B)$ using only one row (similar to Knapsack).

Example 6: Subset Sum of Specific Size

Problem Specification

Given a set $S = \{s_0, \ldots, s_{n-1}\}$, a target sum B, and a target size C. Determine if there exists a subset $S' \subseteq S$ such that $\sum_{x \in S'} x = B$ AND $|S'| = C$.

Recursive Formulation

Specification: Let $T(i, j, k)$ be TRUE if a subset of $\{s_0, \ldots, s_i\}$ summing to j using exactly k elements exists. Goal: Compute $T(n-1, B, C)$.

Recurrence Relation: Consider element s_i. A sum j using k elements from $\{s_0, \ldots, s_i\}$ exists if:

1. The sum j using k elements can be achieved using only $\{s_0, \ldots, s_{i-1}\}$. Corresponds to $T(i-1, j, k)$.
2. OR, element s_i is used (requires $k \geq 1, j \geq s_i$), and the remaining sum $j - s_i$ can be achieved using $k-1$ elements from $\{s_0, \ldots, s_{i-1}\}$. Corresponds to $T(i-1, j-s_i, k-1)$.

$$T(i, j, k) = T(i-1, j, k) \vee (j \geq s_i \wedge k \geq 1 \wedge T(i-1, j-s_i, k-1))$$

Base Cases:

- $T(i, 0, 0) =$ TRUE for all i (empty set sums to 0 with 0 elements).
- $T(-1, j, k) =$ FALSE if $j > 0$ or $k > 0$.
- Initialize based on s_0: $T(0, s_0, 1) =$ TRUE (if $s_0 \leq B$), $T(0, 0, 0) =$ TRUE. All others $T(0, j, k) =$ FALSE.

Proof of Correctness

Similar logic to the standard Subset Sum proof, but carries the subset size k constraint through the induction. The recurrence correctly checks both possibilities (using s_i or not) while ensuring the sum and element count constraints are met based on the assumed correctness of subproblems.

Tabulation Approach

Algorithm:

1. Create a 3D boolean table $T[0..n-1][0..B][0..C]$.
2. Initialize base cases (e.g., $T[i][0][0] = \text{TRUE}$, handle item 0 separately).
3. Iterate i from 1 to $n-1$ (items).
4. Iterate j from 0 to B (sums).
5. Iterate k from 0 to C (sizes). *(Start k from 1 if base case T[..][0][0]=TRUE handles k=0)*
   - Set $T[i][j][k] = T[i-1][j][k]$.
   - If $j \geq s_i$ and $k \geq 1$:
     - $T[i][j][k] = T[i][j][k] \vee T[i-1][j-s_i][k-1]$.
6. Return $T[n-1][B][C]$.

Runtime Analysis:

- Table size is $n \times (B+1) \times (C+1)$.
- **Time Complexity:** $O(nBC)$.
- **Space Complexity:** $O(nBC)$. Can be optimized to $O(BC)$ using only two layers (for i and $i-1$).

Example 7: Longest Common Subsequence (LCS)

Problem Specification

Given two sequences $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$, find the length of the longest subsequence that is common to both X and Y.

Recursive Formulation

Specification: Let $T(i,j)$ be the length of the LCS of the prefixes $X[1\ldots i]$ and $Y[1\ldots j]$. Goal: Compute $T(m,n)$.

Recurrence Relation: Consider the last characters x_i and y_j.

1. **If $x_i = y_j$:** The character $x_i(= y_j)$ must be the last character of the LCS of $X[1..i]$ and $Y[1..j]$. The preceding part of the LCS must be the LCS of $X[1..i-1]$ and $Y[1..j-1]$. Length $= 1 + T(i-1, j-1)$.
2. **If $x_i \neq y_j$:** The LCS cannot end with both x_i and y_j. It must be the LCS of either $X[1..i-1]$ and $Y[1..j]$ (excluding x_i) OR the LCS of $X[1..i]$ and $Y[1..j-1]$ (excluding y_j). We take the longer of these two. Length $= \max(T(i-1,j), T(i,j-1))$.

$$T(i,j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + T(i-1, j-1) & \text{if } x_i = y_j \\ \max(T(i-1,j), T(i,j-1)) & \text{if } x_i \neq y_j \end{cases}$$

Base Cases: $T(i,0) = 0$ for all i, $T(0,j) = 0$ for all j. (LCS with an empty string is empty).

Proof of Correctness

Base Cases: Correct, LCS with empty string has length 0. **Inductive Step:** Assume $T(a, b)$ is correct for all $a \leq i, b \leq j$ where $a + b < i + j$.

- **Case 1 ($x_i = y_j$):** Let Z be an LCS of $X[1..i]$ and $Y[1..j]$. If $z_k = x_i = y_j$ is the last character of Z, then $Z[1..k-1]$ must be an LCS of $X[1..i-1]$ and $Y[1..j-1]$. By hypothesis, $|Z[1..k-1]| = T(i-1, j-1)$, so $|Z| = 1 + T(i-1, j-1)$. If $z_k \neq x_i$, then Z is an LCS of $X[1..i-1]$ and $Y[1..j]$ or $X[1..i]$ and $Y[1..j-1]$. But we could append x_i to the LCS of $X[1..i-1]$ and $Y[1..j-1]$ to get a common subsequence of length $1 + T(i-1, j-1)$, which must be $\geq |Z|$ (by optimality of $T(i-1, j)$ etc.). Thus, the recurrence $1 + T(i-1, j-1)$ is correct.
- **Case 2 ($x_i \neq y_j$):** Let Z be an LCS of $X[1..i]$ and $Y[1..j]$. Since $x_i \neq y_j$, Z cannot end in both. If $z_k \neq x_i$, then Z is a common subsequence of $X[1..i-1]$ and $Y[1..j]$, so $|Z| \leq T(i-1, j)$. If $z_k \neq y_j$, then Z is a common subsequence of $X[1..i]$ and $Y[1..j-1]$, so $|Z| \leq T(i, j-1)$. Thus, $|Z| \leq \max(T(i-1, j), T(i, j-1))$. Furthermore, the LCS of $X[1..i-1]$ and $Y[1..j]$ (length $T(i-1, j)$) and the LCS of $X[1..i]$ and $Y[1..j-1]$ (length $T(i, j-1)$) are both common subsequences of $X[1..i]$ and $Y[1..j]$. Therefore, the maximum possible length is $\max(T(i-1, j), T(i, j-1))$.

The recurrence correctly computes the LCS length.

Tabulation Approach

Algorithm:

1. Create a 2D table $T[0..m][0..n]$.
2. Initialize the 0-th row and 0-th column to 0.
3. Iterate i from 1 to m:
   - Iterate j from 1 to n:
     - If $X[i] = Y[j]$: Set $T[i][j] = 1 + T[i-1][j-1]$.
     - Else ($X[i] \neq Y[j]$): Set $T[i][j] = \max(T[i-1][j], T[i][j-1])$.
4. Return $T[m][n]$.

Reconstructing the LCS: Backtrack from $T[m][n]$.

- If $X[i] = Y[j]$, this character is part of the LCS. Prepend it to the result and move diagonally to $T[i-1][j-1]$.
- If $X[i] \neq Y[j]$, move to the cell that yielded the maximum: If $T[i-1][j] \geq T[i][j-1]$, move up to $T[i-1][j]$; otherwise, move left to $T[i][j-1]$.
- Stop when $i = 0$ or $j = 0$.

Runtime Analysis:

- Table size is $(m+1)(n+1)$. Filling each entry takes $O(1)$.
- **Time Complexity:** $O(mn)$.
- **Space Complexity:** $O(mn)$. Can be optimized to $O(\min(m, n))$ if only the length is needed. Reconstruction typically requires $O(mn)$ space or recomputation.

Greedy Algorithms

Core Idea: Greedy algorithms build a solution step-by-step, making the choice at each stage that appears to be the best according to some local criterion. The hope is that these locally optimal choices will lead to a globally optimal solution for the entire problem.

Contrast with Dynamic Programming:

- Dynamic Programming typically explores multiple options for subproblems and chooses the best one, often storing results in a table. It makes a decision based on optimal solutions to subproblems.
- Greedy Algorithms make a single choice deemed "best" at the moment (the greedy choice) and commit to it, without reconsidering past choices or exploring alternatives extensively. The subproblem resulting from the greedy choice is then solved.

Warning: Greedy algorithms are appealing due to their simplicity and often efficiency, but they do **not** work for all optimization problems. It is crucial to rigorously prove the correctness of a greedy algorithm before trusting its output. When a greedy strategy fails, dynamic programming or other techniques might be necessary.

The Exchange Argument: Proving Greedy Correctness

A common and powerful technique to prove the correctness of a greedy algorithm is the **exchange argument** (or "staying ahead" argument). The general structure is:

1. **Assume an Optimal Solution:** Let OPT be an arbitrary optimal solution to the problem.
2. **Compare with Greedy:** Let GREEDY be the solution generated by the greedy algorithm. Assume GREEDY $\neq$ OPT.
3. **Find First Difference:** Identify the first step or choice where GREEDY differs from OPT. For example, the first item selected, the first decision made.
4. **The "Exchange":** Show that you can modify OPT by replacing its differing choice with the greedy choice made by GREEDY at that step. This modification should result in a new solution OPT$'$ that is:
   - Still valid (satisfies the problem constraints).
   - At least as good as OPT (its objective value is no worse).
   - "Closer" to the greedy solution GREEDY (it now agrees with GREEDY at the point of the first difference).
5. **Iteration:** Argue that this exchange process can be repeated, step-by-step transforming OPT into GREEDY without worsening the objective value.
6. **Conclusion:** Since OPT was optimal, and GREEDY has an objective value at least as good as (and often identical to) OPT, GREEDY must also be optimal.

Example 1: Activity Selection (Interval Scheduling)

Problem Specification

Given n activities, each with a start time s_i and finish time f_i. Select a maximum-size subset of mutually compatible (non-overlapping) activities. Assume activities are sorted by finish time: $f_1 \leq f_2 \leq \ldots \leq f_n$.

Greedy Algorithm

1. Select the first activity a_1 (earliest finish time). Add to solution set S.
2. Let last_finish be the finish time of the most recently selected activity.
3. Iterate through remaining activities a_i (from $i = 2$ to n):
   - If $s_i \geq$ last_finish, select a_i, add to S, and update last_finish $= f_i$.
4. Return S.

Greedy Choice: Always select the compatible activity with the earliest finish time.

Proof of Correctness (Exchange Argument)

1. Let $G = \{g_1, g_2, \ldots, g_k\}$ be the greedy solution (sorted by finish times).
2. Let $O = \{o_1, o_2, \ldots, o_m\}$ be an optimal solution (sorted by finish times). We want to show $k = m$.
3. Assume $G \neq O$. Let j be the first index where $g_j \neq o_j$. This means $g_1 = o_1, \ldots, g_{j-1} = o_{j-1}$.
4. By definition of the greedy algorithm, g_j is the activity with the earliest finish time that starts after g_{j-1} (or o_{j-1}) finishes. Therefore, $f(g_j) \leq f(o_j)$.
5. **Exchange:** Create a new solution $O' = (O \setminus \{o_j\}) \cup \{g_j\}$.
6. **Validity:** Is O' compatible?
   - g_j starts after $o_{j-1} = g_{j-1}$ finishes (by greedy selection).
   - Since $f(g_j) \leq f(o_j)$, g_j finishes no later than o_j. Any subsequent activity o_{j+1} in O started after o_j finished, so it must also start after g_j finishes.
   - Yes, O' is a set of compatible activities.
7. **Quality:** $|O'| = |O| = m$, so O' is also optimal.
8. **Closer to Greedy:** O' now agrees with G up to index j.
9. **Iteration:** We can repeat this exchange for the next index where O' differs from G. Eventually, we transform O into G without decreasing the number of activities.
10. **Conclusion:** Therefore, $|G| = |O| = m$, and the greedy algorithm produces an optimal solution.

Runtime Analysis

- Sorting by finish times (if not already sorted): $O(n \log n)$.
- Greedy selection pass: $O(n)$.
- **Total Time:** $O(n \log n)$.

Example 2: Fractional Knapsack

Problem Specification

Given n items with weights w_i and values v_i, and knapsack capacity W. Items can be taken in fractions ($0 \leq p_i \leq 1$). Maximize total value $\sum p_i v_i$ subject to total weight $\sum p_i w_i \leq W$.

Greedy Algorithm

1. Calculate the value density (value-to-weight ratio) $\rho_i = v_i/w_i$ for each item i.
2. Sort items in descending order based on ρ_i.
3. Initialize total value $V = 0$, current weight $C = 0$.
4. Iterate through sorted items i:
   - If $C + w_i \leq W$: Take the whole item ($p_i = 1$). Update $V += v_i$, $C += w_i$.
   - Else: Take a fraction $p_i = (W - C)/w_i$ of item i. Update $V += p_i v_i$, $C = W$. Break the loop.
5. Return V.

Greedy Choice: Always take as much as possible of the item with the highest remaining value density.

Proof of Correctness (Exchange Argument)

1. Assume items are sorted such that $\rho_1 \geq \rho_2 \geq \ldots \geq \rho_n$.
2. Let $G = (p_1, \ldots, p_n)$ be the greedy solution fractions.
3. Let $O = (q_1, \ldots, q_n)$ be an optimal solution fractions. Assume $G \neq O$.
4. Let i be the first index where $p_i \neq q_i$. Since the greedy algorithm takes as much as possible of items with higher density, it must be that $p_i > q_i$. (Greedy took more of item i than OPT did).
5. Because $\sum p_k w_k = \sum q_k w_k = W$ (assuming knapsack is full for optimality, otherwise add more of highest density item), and greedy took more of item i, there must exist some index $j > i$ where $p_j < q_j$ (OPT took more of a lower-density item j than greedy did). Note $\rho_i \geq \rho_j$.
6. **Exchange:** Consider modifying O to O'. We will "transfer" a small amount of weight $\epsilon > 0$ from item j to item i in the optimal solution O.

- Let $\epsilon_i = \min((p_i - q_i)w_i, (q_j - p_j)w_j)$. This is the amount of weight we can transfer. Let $\epsilon_j = \epsilon_i \frac{w_j}{w_i}$.
- Define $q'_k = q_k$ for $k \neq i, j$.
- Set $q'_i = q_i + \epsilon_i / w_i$ (increase fraction of item i).
- Set $q'_j = q_j - \epsilon_j / w_j = q_j - \epsilon_i / w_i$ (decrease fraction of item j). *Correction: Decrease by weight amount ϵ_j, so fraction decreases by ϵ_j / w_j. Let's transfer amount of item. Take $\delta = \min((p_i - q_i)w_i, (q_j - p_j)w_j)$. Increase fraction of item i by δ / w_i, decrease fraction of item j by δ / w_j.*
- Let's try a simpler way: Decrease amount of item j in O by δw_j and increase amount of item i by δw_i, where $\delta w_i \leq (p_i - q_i)w_i$ and $\delta w_j \leq (q_j - p_j)w_j$. Choose δ small enough. Better: take amount $\epsilon = \min((p_i - q_i)w_i, (q_j - p_j)w_j)$. Replace amount ϵ of item j with amount ϵ of item i.
- New solution O'. Total weight is unchanged: $-\epsilon + \epsilon = 0$.
- Change in value: $\epsilon\rho_i - \epsilon\rho_j = \epsilon(\rho_i - \rho_j) \geq 0$ (since $\rho_i \geq \rho_j$).

7. **Validity & Quality:** The new solution O' is valid (total weight $\leq W$, $0 \leq q'_k \leq 1$) and its value is $\geq$ value of O. So O' is also optimal.

8. **Closer to Greedy:** O' is closer to G because it now takes more of item i and less of item j.

9. **Iteration:** This process can be repeated until the solution matches G.

10. **Conclusion:** G must be optimal.

Runtime Analysis

- Calculate ratios: $O(n)$.
- Sorting by ratio: $O(n \log n)$.
- Filling knapsack pass: $O(n)$.
- **Total Time:** $O(n \log n)$.

Example 3: Magnetic Tape Storage

Problem Specification

Given n files with lengths $L[1..n]$ and access frequencies $F[1..n]$. Find an ordering (permutation π) of files on a tape to minimize the total access cost, where cost is $\sum_{k=1}^{n} \left(F[\pi(k)] \sum_{i=1}^{k} L[\pi(i)] \right)$.

Greedy Algorithm

1. Sort files by the ratio $L[i]/F[i]$ in non-decreasing order.

2. Place files on the tape in this sorted order.

Greedy Choice: Place the file with the smallest L/F ratio next.

Proof of Correctness (Exchange Argument)

1. Assume an optimal permutation π is not sorted by L/F. Then there must exist an adjacent pair $i, i+1$ such that $\frac{L[\pi(i)]}{F[\pi(i)]} > \frac{L[\pi(i+1)]}{F[\pi(i+1)]}$. Let $a = \pi(i)$ and $b = \pi(i+1)$. So $L[a]/F[a] > L[b]/F[b]$, which implies $L[a]F[b] > L[b]F[a]$.
2. **Exchange:** Consider swapping files a and b in the permutation to get π'. Only the costs related to accessing files a and b change, and the costs for accessing files after b change due to the altered prefix sum.
3. Let $P = \sum_{k=1}^{i-1} L[\pi(k)]$ be the sum of lengths before a and b.
4. Cost contribution of a, b in π: $F[a](P + L[a]) + F[b](P + L[a] + L[b])$.
5. Cost contribution of b, a in π': $F[b](P + L[b]) + F[a](P + L[b] + L[a])$.
6. Change in cost = Cost(π') - Cost(π) = $F[b]L[b] + F[a]L[b] + F[a]L[a] - (F[a]L[a] + F[b]L[a] + F[b]L[b])$
7. Change = $F[a]L[b] - F[b]L[a]$.
8. Since we assumed $L[a]F[b] > L[b]F[a]$, the change is $L[b]F[a] - L[a]F[b] < 0$.
9. **Conclusion:** Swapping the out-of-order adjacent pair a, b strictly decreases the total cost, contradicting the optimality of π. Therefore, the optimal permutation must be sorted by non-decreasing L/F ratio. The greedy algorithm achieves this.

Runtime Analysis

- Calculate ratios: $O(n)$.
- Sorting by ratio: $O(n \log n)$.
- **Total Time:** $O(n \log n)$.

(Note: The simple case with unit frequencies $F[i] = 1$ simplifies to sorting by non-decreasing length $L[i]$).

Example 4: Huffman Coding

Problem Specification

Given characters c_i with frequencies f_i. Find a prefix-free binary code (represented by a binary tree) that minimizes the average code length $\sum f_i \cdot \text{depth}(c_i)$.

Greedy Algorithm (Huffman's Algorithm)

1. Create a leaf node for each character c_i with weight f_i.
2. Use a min-priority queue Q to store nodes, prioritized by frequency.
3. While $|Q| > 1$:
    - Extract the two nodes x, y with the minimum frequencies from Q.

- Create a new internal node z with $z.freq = x.freq + y.freq$, $z.left = x$, $z.right = y$.
- Insert z back into Q.

4. The single remaining node in Q is the root of the optimal Huffman tree.

Greedy Choice: Repeatedly merge the two nodes/subtrees with the lowest current frequencies.

Proof of Correctness (Sketch)

Relies on two key properties, often proven using exchange arguments:

1. **Greedy Choice Property:** There exists an optimal prefix code tree where the two characters with the lowest frequencies (x, y) are siblings at the maximum depth. *Proof Sketch (Exchange):* Assume an optimal tree T where x, y are not siblings, but some b, c are siblings at max depth d_{max}. Since f_x, f_y are minimal frequencies, $f_x \leq f_b$ and $f_y \leq f_c$. Also $d(x) \leq d_{max} = d(b)$ and $d(y) \leq d_{max} = d(c)$. Swapping x with b and y with c results in a new tree T'. The change in cost is $(f_b d(b) + f_x d(x)) - (f_b d(x) + f_x d(b)) +$ (similar for y,c). This simplifies to $(f_b - f_x)(d(b) - d(x)) + (f_c - f_y)(d(c) - d(y)) \geq 0$. So the cost of T' is no more than T, and T' has x, y as siblings at max depth.

2. **Optimal Substructure Property:** If we merge the two lowest frequency characters x, y into a meta-character z with frequency $f_x + f_y$, then an optimal tree for the original problem can be constructed from an optimal tree for the reduced problem (with z replacing x, y).

Huffman's algorithm correctly implements this greedy choice and optimal substructure.

Runtime Analysis

- Building initial priority queue: $O(n)$.
- Loop runs $n - 1$ times. Each iteration performs 2 'extract-min' and 1 'insert'.
- Using a binary heap: $O(\log n)$ per operation.
- **Total Time:** $O(n \log n)$.

Other Problems & Considerations

- **Optimal File Merging:** Merge n files with sizes L_i. Merging two files costs the sum of their sizes. Goal: minimize total merge cost. *Greedy solution:* Repeatedly merge the two currently smallest files (use a min-heap). This is structurally identical to Huffman coding. Proof follows similarly. Time: $O(n \log n)$.
- **Minimum Spanning Tree (MST):** Both Prim's and Kruskal's algorithms are greedy.

  - **Prim's:** Greedily adds the cheapest edge connecting the current tree to a vertex outside the tree. Correctness via Cut Property (a direct greedy choice proof).
  - **Kruskal's:** Greedily adds the overall cheapest edge that doesn't form a cycle. Correctness via Cut Property or Cycle Property (exchange argument applicable).

- **Coin Change:** The standard greedy approach (taking the largest denomination possible) works for canonical coin systems (like US currency) but **fails** for arbitrary denominations. E.g., Denominations $\{1, 3, 4\}$, Target 6. Greedy: $4+1+1$ (3 coins). Optimal: $3 + 3$ (2 coins). This requires DP for the general case.

- **Job Sequencing with Deadlines:** Given jobs with profits and deadlines (unit time execution). *Greedy solution:* Sort jobs by decreasing profit. For each job, schedule it in the latest possible available time slot $\leq$ its deadline. Correctness can be shown with an exchange argument. Time: $O(n^2)$ or $O(n \log n)$ with DSU.

- **Container With Most Water:** Uses a two-pointer approach. While sometimes categorized with greedy, it's more about intelligently shrinking the search space based on the limiting factor (the shorter height). The "greedy" choice is which pointer to move. Correctness relies on showing the optimal container cannot be missed by discarding the shorter line's position. Time: $O(n)$.

- **Minimum Platforms / Interval Covering:** These often involve sorting and making a greedy choice based on start/end times or coverage extent. Correctness proofs usually involve exchange arguments showing the greedy choice doesn't preclude an optimal solution.

Example 5: Optimal File Merging

Problem Specification

Given n files with sizes $L_1, L_2, \ldots, L_n$. Merging two files of size a and b costs $a + b$. Find an order of merging pairs of files (or merged files) until only one file remains, such that the total cost of all merges is minimized.

Greedy Algorithm

This problem is structurally equivalent to Huffman coding where file sizes correspond to frequencies.

1. Create a min-priority queue Q and insert all initial file sizes.
2. Initialize total cost $C = 0$.
3. While $|Q| > 1$:
   - Extract the two smallest sizes $a =$ `extract-min`(Q) and $b =$ `extract-min`(Q).
   - Calculate merge cost: cost $= a + b$.
   - Add cost to total: $C = C +$ cost.

- Insert the new merged file size cost back into Q.

4. Return C.

Greedy Choice: Always merge the two smallest currently available files/merged groups.

Proof of Correctness (Exchange Argument - similar to Huffman)

The proof mirrors the Huffman coding proof.

1. **Greedy Choice Property:** An optimal merge sequence exists where the two smallest initial files (L_x, L_y) are merged at some point, effectively becoming siblings in the merge tree. If they aren't merged together first in an optimal solution, swapping them with the actual first pair merged doesn't increase the cost.

2. **Optimal Substructure:** If merging L_x and L_y is the first step, the optimal way to finish the merge corresponds to optimally merging the remaining $n-1$ files (where L_x, L_y are replaced by $L_x + L_y$).

The greedy algorithm implements exactly this process.

Runtime Analysis

- Building initial heap: $O(n)$.
- Loop runs $n-1$ times. Each iteration: 2 'extract-min' ($O(\log n)$), 1 'insert' ($O(\log n)$).
- **Total Time:** $O(n \log n)$.

Example 6: Job Sequencing with Deadlines

Problem Specification

Given n jobs, each with profit $p_i > 0$ and deadline $d_i \geq 1$. Each job takes 1 unit of time. Find a schedule (subset of jobs assigned to time slots $1, 2, \ldots$) such that each scheduled job i finishes by its deadline d_i, and the total profit of scheduled jobs is maximized.

Greedy Algorithm

1. Sort jobs in descending order of profit ($p_1 \geq p_2 \geq \ldots \geq p_n$).

2. Find the maximum deadline $d_{max} = \max(d_i)$.

3. Initialize time slots $1, \ldots, d_{max}$ as free. Keep track of the number of scheduled jobs $count = 0$ and total profit $P = 0$.

4. For each job i from 1 to n (in sorted order):

   - Find the latest possible free slot t such that $1 \leq t \leq d_i$.
   - Iterate t from $\min(d_{max}, d_i)$ down to 1.
   - If a free slot t is found:

  - Schedule job i in slot t. Mark slot t as occupied.
  - Increment *count*, add p_i to P.
  - Break the inner loop (move to the next job).

5. Return the schedule and total profit P.

(Implementation Note: Finding the latest free slot can be optimized using a Disjoint Set Union structure to track blocks of free slots, leading to $O(n \log n)$ or $O(n\alpha(n))$ complexity after sorting).

Greedy Choice: Consider jobs in order of highest profit first. Schedule the current job as late as possible within its deadline.

Proof of Correctness (Exchange Argument Sketch)

1. Let G be the greedy schedule, O be an optimal schedule. Assume $G \neq O$.

2. Let job i be the first job (in the profit-sorted order) where the schedules differ.

3. **Case 1: G schedules i, O does not.** Since O is optimal, it must schedule some other set of jobs O' in the time slots used by G up to job i's deadline. If there's a free slot in O at or before d_i, O could have scheduled i (contradicting optimality since p_i is maximal among remaining jobs). If all slots $\leq d_i$ are full in O, let j be a job scheduled in O at time $t \leq d_i$ that is not scheduled by G. Then $p_j \leq p_i$. We can replace j with i in O to get O' which is still valid and has profit $\geq$ profit of O. This makes O' closer to G.

4. **Case 2: O schedules i, G does not.** This cannot happen because G tries to schedule every job. If i wasn't scheduled by G, it means all slots $\leq d_i$ were already filled by higher-profit jobs. O cannot schedule i and also schedule those higher-profit jobs by time d_i (since they also have deadlines $\geq d_i$), contradicting the feasibility of O.

5. **Case 3: Both schedule i, but in different slots.** Can argue that scheduling i later (as G does) leaves earlier slots free for potentially other high-profit, short-deadline jobs, and doesn't hurt optimality.

Through repeated exchanges, O can be transformed into G without decreasing profit.

Runtime Analysis

- Sorting: $O(n \log n)$.
- Naive slot finding: Each job might scan up to d_{max} slots. Total $O(n \cdot d_{max})$. Can be $O(n^2)$ if $d_{max} \approx n$.
- With DSU optimization: $O(n\alpha(n))$ after sorting.
- **Total Time:** $O(n \log n + n\alpha(n))$ or $O(n \log n)$.

Example 7: Minimum Number of Platforms

Problem Specification

Given arrival times $arr[1..n]$ and departure times $dep[1..n]$ for n trains. Find the minimum number of platforms required at a station so that no train has to wait.

Greedy Algorithm

1. Sort the arrival times: arr'.
2. Sort the departure times: dep'.
3. Initialize $platforms_needed = 1$, $max_platforms = 1$.
4. Initialize pointers $i = 1$ (for next arrival) and $j = 0$ (for next departure).
5. While $i < n$:
   - If $arr'[i] \leq dep'[j]$: A new train arrives before the earliest scheduled departure. Need another platform. Increment $platforms_needed$. Move to next arrival: $i++$.
   - Else ($arr'[i] > dep'[j]$): A train departs before the next arrival. A platform becomes free. Decrement $platforms_needed$. Move to next departure: $j++$.
   - Update $max_platforms = \max(max_platforms, platforms_needed)$.
6. Return $max_platforms$.

Greedy Choice: Process events (arrivals and departures) in chronological order. Increment platform count for an arrival, decrement for a departure. The maximum count encountered is the minimum required.

Proof of Correctness

This isn't a typical exchange argument proof. The algorithm correctly simulates the events chronologically. $platforms_needed$ tracks the number of trains simultaneously present at the station at any time. The maximum value this counter reaches represents the peak demand for platforms. Any fewer platforms would mean at the peak time, some train would have to wait. Thus, the maximum value computed is the minimum required.

Runtime Analysis

- Sorting arrivals: $O(n \log n)$.
- Sorting departures: $O(n \log n)$.
- Two-pointer traversal: $O(n)$.
- **Total Time:** $O(n \log n)$.

Example 8: Interval Covering

Problem Specification

Given a target interval $[T_{start}, T_{end}]$ and a set of n other intervals $\{[s_1, e_1], \ldots, [s_n, e_n]\}$. Find the minimum number of intervals from the set required to completely cover the target interval.

Greedy Algorithm

1. Sort the intervals based on their start times s_i.
2. Initialize $count = 0$, current coverage end $C_{end} = T_{start}$, current index $i = 0$.
3. While $C_{end} < T_{end}$ and $i < n$:
   - Find the interval among those starting at or before C_{end} ($s_k \leq C_{end}$) that extends farthest to the right (maximizes e_k). Let this farthest reach be max_reach.
   - If no such interval exists (or $max_reach \leq C_{end}$), the target cannot be covered. Return -1.
   - Select the interval that achieves max_reach. Increment $count$.
   - Update $C_{end} = max_reach$.
   - Advance i past all intervals considered in this step.
4. If $C_{end} \geq T_{end}$, return $count$. Otherwise, return -1.

Greedy Choice: At each step, extend the current coverage as far right as possible using a single interval that starts at or before the current coverage end.

Proof of Correctness (Exchange Argument Sketch)

1. Let $G = \{g_1, \ldots, g_k\}$ be the greedy solution intervals.
2. Let $O = \{o_1, \ldots, o_m\}$ be an optimal solution (minimum number of intervals), sorted perhaps by start or end times. We want $k = m$.
3. Consider the first interval g_1. It covers $[T_{start}, C^g_{end,1}]$ where $C^g_{end,1}$ is the farthest reach from T_{start}. Let o_1 be the first interval in O. It must cover T_{start}, so $s(o_1) \leq T_{start}$. Its end $e(o_1)$ cannot be greater than $C^g_{end,1}$ by the greedy choice.
4. **Exchange:** Replace o_1 with g_1 in O to get O'. Since $e(g_1) \geq e(o_1)$, g_1 provides at least as much coverage as o_1. The rest of O can potentially cover the remaining part $[e(g_1), T_{end}]$. $|O'| = |O| = m$, and O' is now closer to G.
5. Repeat this process. At each stage, the greedy interval extends coverage at least as far as the corresponding interval in the modified optimal solution. This ensures the greedy approach uses no more intervals than the optimal one.

Runtime Analysis

- Sorting intervals: $O(n \log n)$.
- Greedy selection: In the worst case, each interval might be considered in the inner loop to find the max-reach. If implemented carefully (advancing i), the total time for the while loop is $O(n)$.
- **Total Time:** $O(n \log n)$.

Graph Algorithms

Graphs are fundamental mathematical structures used to model relationships between objects. They consist of vertices (nodes) representing the objects and edges representing connections or relationships between them.

Introduction and Motivation

- **Definition:** A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E. Each edge connects one or two vertices (its endpoints).
  - **Undirected Graph:** Edges are unordered pairs $\{u, v\}$.
  - **Directed Graph (Digraph):** Edges are ordered pairs (u, v), representing a connection from u to v.
  - **Weighted Graph:** Edges have associated weights or costs.
- **Motivation:** Graphs model countless real-world and abstract scenarios.
  - **Networks:** Social networks (people and friendships), transportation networks (cities and routes), computer networks (devices and connections), the World Wide Web (pages and hyperlinks).
  - **Dependencies:** Program modules, task precedence, citation networks.
  - **States and Transitions:** State spaces in puzzles (Rubik's Cube, mazes), game trees, finite automata.
  - **Relationships:** Collaboration graphs, protein interactions, ecological niches.
- **Historical Context (Euler):** The Königsberg Bridge Problem, solved by Leonhard Euler in 1736, is considered the birth of graph theory. Euler determined that a walk traversing each bridge (edge) exactly once was impossible because not all landmasses (vertices) had an even number of bridges connected to them (even degree). This led to the concept of Eulerian paths and circuits.
  - An **Eulerian path** traverses each edge exactly once.
  - An **Eulerian circuit** is an Eulerian path that starts and ends at the same vertex.
  - A connected graph has an Eulerian circuit if and only if every vertex has an even degree.
  - A connected graph has an Eulerian path if and only if it has exactly zero or two vertices of odd degree.

Basic Terminology and Properties

- **Adjacent Vertices (Neighbors):** Two vertices connected by an edge.
- **Degree (Undirected):** $\deg(v)$ = number of edges incident on vertex v. Loops count twice.
- **In-degree (Directed):** $\deg^-(v)$ = number of edges entering v.
- **Out-degree (Directed):** $\deg^+(v)$ = number of edges leaving v.
- **Path:** A sequence of vertices $v_0, v_1, \ldots, v_k$ such that $\{v_{i-1}, v_i\}$ (undirected) or (v_{i-1}, v_i) (directed) is an edge for all $i = 1..k$.
- **Cycle:** A path that starts and ends at the same vertex and has distinct edges.
- **Connected Graph (Undirected):** A path exists between any two distinct vertices.
- **Strongly Connected Graph (Directed):** A directed path exists from u to v and from v to u for all pairs u, v.
- **Subgraph:** A graph $H = (W, F)$ is a subgraph of $G = (V, E)$ if $W \subseteq V$ and $F \subseteq E$.
- **Induced Subgraph:** Given $W \subseteq V$, the induced subgraph contains all edges from E whose endpoints are both in W.
- **Handshaking Lemma:**
  - Undirected: $\sum_{v \in V} \deg(v) = 2|E|$. (Corollary: The number of odd-degree vertices is even).
  - Directed: $\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|$.
- **Edge Bounds (Simple Graphs):**
  - Undirected: $0 \leq |E| \leq \binom{|V|}{2} = \frac{|V|(|V|-1)}{2}$.
  - Directed: $0 \leq |E| \leq |V|(|V| - 1)$.

Graph Representations

How we store a graph affects algorithm efficiency.

Adjacency List

Description: An array (or map) where the entry for vertex u contains a list of its adjacent vertices. For weighted graphs, store pairs $(v, \text{weight}(u, v))$.

Vertex	Adjacent Vertices
a	$\rightarrow [b, c, e]$
b	$\rightarrow [a]$
c	$\rightarrow [a, d, e]$
d	$\rightarrow [c, e]$
e	$\rightarrow [a, c, d]$

Analysis:

- **Space:** $O(|V| + |E|)$.
- **Check edge** (u, v)**:** $O(\deg(u))$.
- **Iterate neighbors of** u**:** $O(\deg(u))$.
- **Pros:** Space-efficient for sparse graphs ($|E| \ll |V|^2$). Efficient neighbor iteration.
- **Cons:** Slower edge checking.

Adjacency Matrix

Description: A $|V| \times |V|$ matrix M where $M[i][j] = 1$ (or weight) if edge (v_i, v_j) exists, and 0 (or ∞) otherwise. For undirected graphs, the matrix is symmetric.

$$\begin{pmatrix} & a & b & c & d & e \\ a & 0 & 1 & 1 & 0 & 1 \\ b & 1 & 0 & 0 & 0 & 0 \\ c & 1 & 0 & 0 & 1 & 1 \\ d & 0 & 0 & 1 & 0 & 1 \\ e & 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Analysis:

- **Space:** $O(|V|^2)$.
- **Check edge** (u, v)**:** $O(1)$.
- **Iterate neighbors of** u**:** $O(|V|)$.
- **Pros:** Fast edge checking/addition/removal. Can be good for dense graphs.
- **Cons:** High space usage for sparse graphs. Slow neighbor iteration.

Other Representations

- **Incidence Matrix:** $|V| \times |E|$ matrix indicating which edges are incident on which vertices.
- **Edge List:** A simple list of pairs (u, v) representing edges. Often used as input format or with algorithms like Kruskal's.

Special Types of Graphs

- **Complete Graph K_n:** Simple undirected graph with an edge between every pair of distinct vertices. Has $\binom{n}{2}$ edges.
- **Cycle Graph C_n:** n vertices connected in a simple cycle $(v_1 - v_2 - \ldots - v_n - v_1)$. Has n edges.
- **Hypercube Q_n:** Vertices are n-bit binary strings. Edge between two vertices if they differ in exactly one bit position. Has 2^n vertices and $n2^{n-1}$ edges.
- **Bipartite Graph:** Vertices can be partitioned into two disjoint sets V_1, V_2 such that every edge connects a vertex in V_1 to one in V_2. Equivalently, contains no odd-length cycles.
- **Complete Bipartite Graph $K_{m,n}$:** Bipartite graph with partitions of size m and n, containing all possible mn edges between the partitions.
- **Tree:** A connected, undirected graph with no cycles. Has $|V| - 1$ edges.
- **Forest:** An undirected graph with no cycles (a collection of disjoint trees). A forest with k connected components has $|V| - k$ edges.
- **Directed Acyclic Graph (DAG):** A directed graph with no directed cycles. Every DAG has at least one source (in-degree 0) and at least one sink (out-degree 0).

Graph Traversal Algorithms

Systematic ways to visit vertices and edges of a graph, starting from a source vertex s. Assume adjacency list representation for complexity analysis unless stated otherwise.

Breadth-First Search (BFS)

Idea: Explore the graph layer by layer. Find all vertices reachable from s and compute the shortest path (in number of edges) from s to each reachable vertex. Uses a FIFO (First-In, First-Out) queue.

Algorithm:

1. Initialize:
    - Mark all vertices v as unvisited (`color[v]=WHITE`).
    - Set distance `dist[v]` $= \infty$, predecessor `parent[v]` = NIL.

2. Mark source s: `color[s]=GRAY`, `dist[s]=0`.
3. Initialize queue Q and enqueue s.
4. While Q is not empty:
   - Dequeue vertex u from Q.
   - For each neighbor v of u ($v \in \text{Adj}[u]$):
     - If `color[v] == WHITE`:
       * Mark v as visiting: `color[v]=GRAY`.
       * Set distance: `dist[v]` $=$ `dist[u]` $+ 1$.
       * Set predecessor: `parent[v]` $= u$.
       * Enqueue v.
   - Mark u as finished: `color[u]=BLACK`.

Properties & Correctness:

- Visits vertices in increasing order of distance from s.
- Correctly computes shortest path distances (`dist`$[v]$) in unweighted graphs. Proof involves induction on distance: BFS discovers all vertices at distance k before discovering any vertex at distance $k + 1$.
- The `parent` pointers form a **BFS Tree** rooted at s, containing shortest paths from s.
- Marks exactly the vertices reachable from s.

Runtime Analysis:

- Each vertex is enqueued and dequeued at most once: $O(|V|)$.
- Each edge (u, v) is examined at most twice (once from u, once from v if undirected) when its endpoints are dequeued: $O(|E|)$.
- **Total Time:** $O(|V| + |E|)$.

Depth-First Search (DFS)

Idea: Explore as deeply as possible along one path before backtracking. Uses a LIFO (Last-In, First-Out) stack, often implemented implicitly via recursion.

Algorithm (Recursive):

```
DFS_Outer(G):
  for each vertex u in G.V:
    u.color = WHITE
    u.parent = NIL
  // Optional: time = 0 (for discovery/finish times)
  for each vertex u in G.V:
    if u.color == WHITE:
      DFS_Visit(G, u)
```

```
DFS_Visit(G, u):
  // Optional: time = time + 1; u.discovery_time = time
  u.color = GRAY
  for each v in G.Adj[u]: // Explore edge (u,v)
    if v.color == WHITE:
      v.parent = u
      DFS_Visit(G, v)
    // Optional: handle GRAY (back edge) or BLACK neighbors
  u.color = BLACK
  // Optional: time = time + 1; u.finish_time = time
```

Algorithm (Iterative with Stack):

1. Initialize stack S. Push start vertex s.
2. Initialize 'visited' set/array.
3. While S is not empty:
   - Pop vertex u from S.
   - If u is not visited:
     - Mark u as visited. (Process u).
     - For each neighbor v of u:
       - If v is not visited, push v onto S.

(Note: Iterative version processes nodes in a slightly different order than recursive; recursive explores fully before marking BLACK)

Properties:

- Explores branches deeply.
- The `parent` pointers form a **DFS Forest** (one tree for each starting call to $\text{DFS}_{\text{Visit}}$ in $\text{DFS}_{\text{Outer}}$).
- Useful for cycle detection, topological sorting (DAGs), finding strongly connected components (directed graphs).
- **Discovery/Finish Times (Optional but useful):** Record when a vertex is first discovered (GRAY) and when exploration from it finishes (BLACK). These times exhibit a parenthesis structure crucial for some applications.
- **Edge Classification (based on v's color when exploring edge (u, v))**: Tree edge (WHITE), Back edge (GRAY - indicates cycle), Forward/Cross edge (BLACK).

Runtime Analysis:

- $\text{DFS}_{\text{Visit}}$ is called once for each vertex: $O(|V|)$.
- Each edge (u, v) is examined once (from u) in directed, or twice in undirected: $O(|E|)$.
- **Total Time:** $O(|V| + |E|)$.

Applications & Reductions to Graph Search

BFS and DFS are fundamental building blocks for many graph algorithms and problem-solving techniques. Many problems can be solved by modeling them as graphs and applying a suitable search.

Connectivity and Reachability

- Running BFS or DFS from a source s finds all vertices reachable from s.
- To find connected components in an undirected graph, repeatedly start BFS/DFS from an unvisited vertex until all vertices are visited. Each run identifies one component. Time: $O(|V| + |E|)$.

Shortest Paths (Unweighted Graphs)

- BFS finds the shortest path in terms of the number of edges from source s to all reachable vertices v. The distance is stored in `dist`$[v]$.
- The path can be reconstructed by backtracking using the `parent` pointers from v to s.

Cycle Detection

- **Undirected Graphs:** During BFS or DFS, if we explore edge (u, v) and find that v is already visited and v is not the immediate parent of u in the current search tree, a cycle exists.
- **Directed Graphs:** A cycle exists if and only if DFS finds a **back edge** (an edge (u, v) where v is an ancestor of u in the DFS tree, identified because v is GRAY when (u, v) is explored).
- **Example: Cycle in 2D Grid:** Model the grid as a graph where cells with the same character are vertices and edges connect adjacent cells. Run DFS/BFS. If we encounter a visited neighbor (that isn't the immediate predecessor in the path) with the same character, a cycle exists. Need to track the parent in the search path to avoid immediate backtracking counting as a cycle.

Bipartite Testing

- **Property:** A graph is bipartite iff it has no odd-length cycles.
- **Algorithm:** Use BFS and 2-coloring. Start BFS from a source s, color it (say, level 0 = RED). Color its neighbors (level 1) BLUE. Color their uncolored neighbors (level 2) RED, etc. During exploration, if an edge (u, v) connects two vertices of the *same* color, the graph is not bipartite (an odd cycle has been found). If the BFS completes without conflict, the graph component is bipartite. Repeat for all components. Time: $O(|V| + |E|)$.
- **Disjoint Vertex Covers:** A graph has two disjoint vertex covers iff it is bipartite. The two color classes from a successful 2-coloring form the disjoint vertex covers.

Tree Diameter

- **Algorithm (for Trees only):** 1. Run BFS starting from an arbitrary node s. Find the node u farthest from s. 2. Run BFS starting from u. Find the node v farthest from u. 3. The distance $\texttt{dist}(u, v)$ found in the second BFS is the diameter of the tree. The path can be reconstructed.
- **Correctness:** Relies on the property that an endpoint of any longest path (diameter) in a tree must be the node farthest from any other given node.
- Time: Two BFS runs $= O(|V| + |E|) = O(|V|)$ since $|E| = |V| - 1$ for trees.

Topological Sort (DAGs)

- Application of DFS. A linear ordering of vertices such that for every directed edge (u, v), u comes before v. Possible iff the graph is a DAG.
- Algorithm: Run DFS. As each vertex finishes (is colored BLACK), prepend it to a list. The final list is a topological sort. (Details in a separate DP/Graphs section potentially).

Strongly Connected Components (Directed Graphs)

- Application of DFS (Kosaraju's or Tarjan's algorithm). Partition vertices into maximal sets where any two vertices in a set are mutually reachable. (Details later).

Reduction: State Space Search (Puzzles)

Many puzzles can be solved by modeling their configurations as states (vertices) and moves as transitions (edges). BFS finds the shortest sequence of moves.

- **Fill Coloring:** Vertices = grid cells with value 0. Edges = between adjacent 0-cells. BFS/DFS finds the connected component reachable from the start cell.
- **Mazes:** Vertices = empty cells. Edges = between adjacent empty cells. BFS/DFS finds a path from start to end. BFS finds the shortest path.
- **Number Maze:** Vertices = (i, j) grid cells. Edges = $(i, j) \to (i', j')$ if a valid move exists based on value $A[i][j]$. BFS finds the minimum number of moves.
- **Snakes and Ladders:** Vertices = squares $1..N$. Edges = move k steps ahead ($1 \leq k \leq$ dice) + follow snake/ladder if present. BFS finds the minimum number of dice rolls.
- **Vidrach Itky Leda:** Vertices = states (pos_{red}, pos_{blue}). Edges = valid moves according to rules. BFS finds minimum moves to swap.
- **Word Ladder:** Vertices = valid words. Edges = between words differing by one letter. BFS finds the shortest transformation sequence.
- **Knight's Shortest Path:** Vertices = chessboard squares. Edges = valid knight moves. BFS finds the minimum number of moves.
- **Rubik's Cube:** Vertices = cube configurations. Edges = quarter turns. BFS finds minimum moves ("God's Number"). State space is huge!

Reduction: State Graph Transformations

Sometimes, the state needs to include more information than just the location.

- **Alternating Color Path:** Given edges colored Red/Blue. Find path with alternating edge colors. *State Graph:* Vertices = $(u, \text{last_color_used})$. Example: (u, RED) means arrived at u via a red edge. *Edges:* If (u, v) is BLUE, add edge $(u, \text{RED}) \to (v, \text{BLUE})$. If (u, v) is RED, add edge $(u, \text{BLUE}) \to (v, \text{RED})$. (Need initial states too). Find path in this state graph using BFS/DFS.
- **Walk Length Divisible by 3:** Find walk $s \to t$ with length $L \equiv 0 \pmod 3$. *State Graph:* Vertices = $(u, \text{length} \pmod 3)$. *Edges:* If (u, v) exists in original graph, add edges $(u, r) \to (v, (r + 1) \pmod 3)$ for $r = 0, 1, 2$ in state graph. *Goal:* Find path from $(s, 0)$ to $(t, 0)$ using BFS/DFS.

Other Problems

- **Minimum Roads Problem:** Requires connecting n cities such that any two are ≤ 2 roads apart, avoiding $m < n/2$ forbidden pairs. Solution involves finding a vertex c not involved in any forbidden pair (guaranteed to exist since $m < n/2$) and making it a central hub (a star graph). Connect c to all other $n - 1$ cities. Minimum roads $= n-1$. This relies on graph structure properties, not direct search.
- **Map Coloring:** Assigning colors to regions on a map (vertices) such that no adjacent regions (sharing a border/edge) have the same color. The Four Color Theorem states 4 colors suffice for planar graphs. Finding minimum colors (chromatic number) is NP-hard.

Graph Traversal: Applications and Reductions

Graph traversal algorithms like Breadth-First Search (BFS) and Depth-First Search (DFS) are fundamental tools for exploring graph structures. Their systematic nature allows them to solve a wide variety of problems directly or serve as subroutines within more complex algorithms. Furthermore, many seemingly unrelated problems can be solved efficiently by modeling them as graph problems and applying BFS or DFS.

Core Applications

Reachability and Connectivity

- Running BFS or DFS starting from a source vertex s identifies all vertices reachable from s.
- **Connected Components (Undirected):** To find all connected components, initialize all vertices as unvisited. Repeatedly start a BFS or DFS from an arbitrary unvisited vertex; all vertices visited in that search form one component. Continue until all vertices are visited. Total time: $O(|V| + |E|)$.
- **Example: Number of Islands:** Given a 2D grid of land ('1') and water ('0'), find the number of islands (connected groups of '1's). Model as a graph where land

cells are vertices and edges connect adjacent land cells. The number of islands is the number of connected components found by running BFS/DFS starting from unvisited land cells.

Shortest Paths (Unweighted Graphs)

- BFS explores the graph layer by layer, guaranteeing that it finds the shortest path in terms of the number of edges from the source s to all other reachable vertices v. The distance is stored as `dist`$[v]$.
- **BFS Tree:** The `Parent` pointers maintained during BFS (`Parent`$(w) = v$ when w is first reached from v) form a tree rooted at s.
- **Path Reconstruction:** To find the shortest path from s to v, follow the `Parent` pointers backward from v until s is reached. Reversing this sequence gives the shortest path.

Cycle Detection

- **Undirected Graphs:** A cycle exists if, during BFS or DFS exploration from edge (u, v), vertex v is found to be already visited and v is not the immediate parent of u in the current search tree.
- **Directed Graphs:** A cycle exists if and only if DFS encounters a **back edge** – an edge (u, v) where v is currently on the recursion stack (marked GRAY in the recursive DFS formulation), meaning v is an ancestor of u.
- **Example: Cycle in 2D Grid:** Given a grid of characters, find if a cycle of adjacent cells with the same character exists (length ≥ 4). Model as a graph. Run DFS starting from each cell. During DFS from cell (i, j) to neighbor (x, y) (with same character): if (x, y) is already visited and is not the immediate parent cell from which we arrived at (i, j), a cycle is found. Keep track of visited status and parent for each DFS path.

Bipartite Testing

- **Property:** An undirected graph is bipartite if and only if it contains no odd-length cycles.
- **Algorithm (BFS 2-Coloring):** 1. Start BFS from an uncolored vertex s, assign it color 1. 2. Assign its neighbors color 2. Assign their uncolored neighbors color 1, and so on (color level k with color $(k \pmod 2) + 1$). 3. During the process, if an edge (u, v) is found where u and v have already been assigned the *same* color, then an odd cycle exists, and the graph is not bipartite. 4. If the BFS completes for a component without conflict, that component is bipartite. Repeat for all components.
- **Time Complexity:** $O(|V| + |E|)$.
- **Disjoint Vertex Covers:** A graph has two disjoint vertex covers if and only if it is bipartite. The two sets of vertices formed by the 2-coloring are the disjoint vertex covers.

Tree Diameter

- **Problem:** Find the longest path between any two nodes in a tree.
- **Algorithm (Two BFS runs):** 1. Start BFS from an arbitrary node s; find the node u with the maximum distance from s. 2. Start BFS from node u; find the node v with the maximum distance from u. 3. The path between u and v is a diameter of the tree, and its length is the distance found in the second BFS.
- **Time Complexity:** $O(|V|)$ for trees.

Topological Sorting (DAGs)

- **Problem:** Find a linear ordering of vertices in a DAG such that for every directed edge (u, v), vertex u comes before vertex v in the ordering.
- **DFS-based Algorithm:** Run DFS. As each vertex v finishes (all its neighbors explored, v becomes BLACK), prepend v to a linked list. The final list is a valid topological sort. Correctness follows from the property that for any edge (u, v) in a DAG, $f[v] < f[u]$ (finish time).
- **Time Complexity:** $O(|V| + |E|)$.

Strongly Connected Components (SCCs)

- **Problem (Directed Graphs):** Partition vertices into maximal sets such that any two vertices within a set are mutually reachable.
- **Kosaraju's Algorithm:** Uses two DFS passes. First DFS on G computes finish times. Second DFS on the transpose graph G^T, processing vertices in decreasing order of finish times from the first pass. Each tree in the second DFS forest is an SCC.
- **Tarjan's Algorithm:** Uses one DFS pass, maintaining discovery times and "low-link" values to identify SCCs using a stack.
- **Time Complexity:** $O(|V| + |E|)$ for both.

Bridge Finding

- **Problem:** Find edges (bridges) in an undirected graph whose removal increases the number of connected components.
- **DFS-based Algorithm:** Run DFS, computing discovery time `disc`$[v]$ and low-link value `low`$[v]$ (lowest discovery time reachable from v via tree edges and at most one back edge). An edge (u, v) (where v is discovered from u) is a bridge if `low`$[v] >$ `disc`$[u]$.
- **Time Complexity:** $O(|V| + |E|)$.

Graph Cloning

- **Problem:** Create a deep copy of a graph given a starting node.
- **Algorithm (BFS/DFS + Hashing):** Use BFS or DFS to traverse the original graph. Maintain a hash map map[original$_{node]}$ $\rightarrow$ cloned$_{\text{node}}$. When visiting an original node ‘u‘, if its clone doesn't exist, create it and store it in the map. For each neighbor ‘v‘ of ‘u‘, recursively clone ‘v‘ (if not already cloned) and add an edge between ‘clone(u)‘ and ‘clone(v)‘.
- **Time Complexity:** $O(|V| + |E|)$.

Reductions: State Space Search via BFS/DFS

Many puzzles and problems can be solved by: 1. Defining the possible configurations or situations as **states** (vertices). 2. Defining valid transitions or moves between states as **edges**. 3. Applying graph search on this implicit **state space graph**. BFS is typically used to find the *shortest sequence* of moves.

- **Fill Coloring Pixel Maps:** States = 0-valued cells. Edges = adjacency between 0-cells. Goal: Find all reachable states from start state (connected component). Use BFS or DFS.
- **Mazes:** States = empty grid cells. Edges = between adjacent empty cells. Goal: Find path from start state to end state. BFS finds shortest path.
- **Number Maze:** States = grid cells (i, j). Edges = valid moves $(i, j) \rightarrow (i \pm A[i][j], j)$ etc. Goal: Find shortest path from top-left to bottom-right. Use BFS.
- **Snakes and Ladders:** States = squares $1..N$. Edges = move k steps ($1 \leq k \leq$ dice) + follow snake/ladder. Goal: Find shortest path (min dice rolls) from 1 to N. Use BFS.
- **Vidrach Itky Leda Puzzle:** States = (pos_{red}, pos_{blue}). Edges = valid moves based on rules. Goal: Find shortest path to swapped state. Use BFS.
- **Word Ladder:** States = words in dictionary. Edges = between words differing by one letter. Goal: Find shortest path from start word to end word. Use BFS.
- **Knight's Shortest Path:** States = chessboard squares (r, c). Edges = valid knight moves. Goal: Find shortest path from start to target. Use BFS.
- **Rotting Oranges:** States = grid configurations. This is naturally solved using BFS starting from all initially rotten oranges simultaneously (multi-source BFS). The distance in BFS corresponds to the time taken. Check if all fresh oranges were reached.
- **Walls and Gates:** States = empty rooms (‘INF‘). Start multi-source BFS from all gates ('0'). Distance in BFS is the distance to the nearest gate.

Reductions: State Graph Transformations

For some problems, the state needs to encode more than just position.

- **Alternating Color Path (Red/Blue Edges):** *State Graph:* Vertices $= (u, \text{last_color_state})$, where state is e.g., START, ARRIVED_VIA_RED, ARRIVED_VIA_BLUE. *Edges:* If (u, v) is BLUE, add edge $(u, \text{ARRIVED_VIA_RED}) \to (v, \text{ARRIVED_VIA_BLUE})$ and $(u, \text{START}) \to (v, \text{ARRIVED_VIA_BLUE})$. Similarly for RED edges. *Goal:* Find path in the state graph using BFS/DFS.
- **Walk Length Divisible by 3 ($s \to t$):** *State Graph:* Vertices $= (u, L \pmod 3)$, where L is the length of the walk to reach u. *Edges:* If (u, v) exists in the original graph, add edges $(u, r) \to (v, (r + 1) \pmod 3)$ for $r = 0, 1, 2$ in the state graph. *Goal:* Check reachability from state $(s, 0)$ to state $(t, 0)$ using BFS/DFS.

Minimum Spanning Trees (MST)

Problem Definition

Given a connected, undirected, weighted graph $G = (V, E, w)$, where $w(e)$ is the weight of edge e, a **Spanning Tree** is a subgraph $T = (V, E')$ such that T is a tree (connected, acyclic) and includes all vertices of G. A **Minimum Spanning Tree (MST)** is a spanning tree whose total edge weight $w(T) = \sum_{e \in E'} w(e)$ is minimized among all possible spanning trees.

Uniqueness: If all edge weights in G are distinct, the MST is unique. (Proof involves assuming two MSTs exist and using cycle properties/exchange arguments).

Generic MST Algorithm and Cut Property

Most MST algorithms build the tree incrementally by adding "safe" edges.

- **Safe Edge:** An edge (u, v) is safe for a set of edges A (which is a subset of some MST) if $A \cup \{(u, v)\}$ is also a subset of some MST.
- **Cut:** A partition of V into two disjoint sets S and $V \setminus S$.
- **Crossing Edge:** An edge with one endpoint in S and the other in $V \setminus S$.
- **Cut Property (Fundamental Theorem):** For any cut $(S, V \setminus S)$, let (u, v) be a minimum-weight edge crossing the cut. Then (u, v) is a safe edge.
- **Generic Algorithm Idea:** Maintain a forest F. Repeatedly find a safe edge that connects two different components in F and add it, until only one component remains.

Prim's Algorithm

Idea: Grow the MST from a single starting vertex s. Maintain a set of vertices S already included in the growing tree. At each step, add the minimum-weight edge connecting a vertex in S to a vertex in $V \setminus S$. Uses a priority queue to efficiently find this edge.

Algorithm:

1. Initialize: `key`$[v] = \infty$ for all v, `parent`$[v]$ = NIL, `inMST`$[v]$ = false. Pick start vertex s, set `key`$[s] = 0$.
2. Initialize min-priority queue Q containing all vertices, prioritized by `key` values.
3. While Q is not empty:
   - Extract vertex u with minimum `key` from Q. Set `inMST`$[u]$ = true.
   - If `parent`$[u] \neq$ NIL, add edge (`parent`$[u], u$) to the MST edges.
   - For each neighbor v of u:
     - If `inMST`$[v]$ == false and $w(u, v) <$ `key`$[v]$:
       * Set `parent`$[v] = u$.
       * Update `key`$[v] = w(u, v)$.
       * Decrease key of v in Q.

Proof Idea: Directly implements the generic approach using the cut $(S, V \setminus S)$, where S is the set of vertices already added to the MST (`inMST=true`). The edge extracted via the priority queue is always the minimum-weight edge crossing this cut, hence it's safe by the Cut Property.

Data Structures & Complexity:

- **Binary Heap:** V extract-min ($O(V \log V)$), E decrease-key ($O(E \log V)$). Total: $O(E \log V)$.
- **Fibonacci Heap:** V extract-min ($O(V \log V)$ amortized), E decrease-key ($O(E)$ amortized). Total: $O(E + V \log V)$.

Kruskal's Algorithm

Idea: Build the MST by considering edges in increasing order of weight and adding an edge only if it connects two previously disconnected components (i.e., doesn't form a cycle). Uses a Disjoint Set Union (DSU) or Union-Find data structure.

Algorithm:

1. Initialize MST edge set $A = \emptyset$.
2. Create a DSU structure with each vertex in its own set ($|V|$ calls to `Make-Set`).
3. Sort all edges E by non-decreasing weight w.
4. For each edge $(u, v) \in E$ in sorted order:
   - If `Find-Set`$(u) \neq$ `Find-Set`(v): *(Checks if u and v are in different components)*
     - Add edge (u, v) to A.
     - Merge the components: `Union`(u, v).
5. Return A.

Proof Idea: When considering edge (u, v), if u and v are in different components C_u, C_v, then (u, v) is the minimum-weight edge crossing the cut $(C_u, V \setminus C_u)$ among edges not yet considered (due to sorting). By the Cut Property, (u, v) is safe to add. Adding it doesn't create a cycle because u and v were in different components.

Data Structures & Complexity:

- Sorting Edges: $O(E \log E)$.
- DSU Operations: $|V|$ Make-Set, $2|E|$ Find-Set, $|V| - 1$ Union. With path compression and union by rank/size, total DSU time is $O(E\alpha(V))$, where α is the very slow-growing inverse Ackermann function (nearly constant).
- **Total Time:** $O(E \log E)$ or $O(E \log V)$, dominated by sorting.

Borůvka's Algorithm

Idea: Works in stages. In each stage, every component identifies its minimum-weight outgoing edge and adds it to the MST forest. Components are then merged based on the added edges.

Algorithm Steps:

1. Initialize forest F with $|V|$ components (each vertex is a component).
2. While F has more than one component:
   - For each component C in F: Find the minimum-weight edge $e_C = (u, v)$ connecting a vertex $u \in C$ to a vertex $v \notin C$.
   - Add all such minimum-weight edges e_C found in this stage to F. (Handle duplicates if multiple components choose the same edge).
   - Update components based on added edges (effectively contracting components connected by the chosen edges).

Complexity and Efficiency:

- Each stage reduces the number of components by at least half. Thus, there are $O(\log V)$ stages.
- Finding the minimum outgoing edge for all components can be done in $O(E)$ time per stage.
- **Total Time:** $O(E \log V)$.
- Well-suited for parallel implementation as minimum edges for components can be found concurrently.

Applications and Related Problems

MST algorithms are used when the goal is to connect all points/nodes with minimum cost.

- **Network Design:** Designing least-cost networks (electricity, plumbing, roads) to connect multiple locations (e.g., "Rebuild the City"). Model locations as vertices, possible connections as edges with costs. The MST gives the minimum cost connection plan.
- **Optimize Water Distribution in a Village:** *Reduction:* Create a graph where houses are nodes. Add a virtual source node representing the main water supply. Add edges from the source to each house i with weight = cost of building a well at house i. Add edges between houses u, v with weight = cost of pipe between u, v. The MST of this augmented graph gives the minimum cost to supply water to all houses (either via a well connected to the source or via pipes from other houses).
- **Clustering:** Can be used as a step in some clustering algorithms (e.g., finding clusters by removing heaviest edges from MST).
- **Earliest Moment When Everyone Becomes Friends:** Given friendship logs `[timestamp, A, B]`. Find the earliest time all n people are in a single connected component. *Reduction:* Sort logs by timestamp. Process logs chronologically using a DSU structure. For each log, perform `Union`(A, B). Maintain a count of disjoint sets. When the count reaches 1, the current log's timestamp is the answer. This is closely related to Kruskal's algorithm's process. Time: $O(E \log E + E\alpha(V))$.

Shortest Path Algorithms

Finding the path with the minimum total weight (cost, distance, time) between vertices in a weighted graph is a fundamental problem with numerous applications. We consider two main variants: Single-Source Shortest Paths (SSSP) and All-Pairs Shortest Paths (APSP).

Core Concepts:

- **Path Weight:** For a path $P = (v_0, v_1, \ldots, v_k)$, the weight is $w(P) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$.
- **Shortest Path Weight $\delta(u, v)$:** $\delta(u, v) = \min\{w(P) \mid P \text{ is a path from } u \text{ to } v\}$. If no path exists, $\delta(u, v) = \infty$. If negative cycles reachable from u can reach v, $\delta(u, v)$ might be $-\infty$.
- **Shortest Path Estimate $d[v]$:** Maintained by algorithms, representing the current best-known distance from the source to v. Initially ∞, converges to $\delta(s, v)$.
- **Predecessor $\pi[v]$:** Stores the vertex preceding v on the current best path found from the source. Used to reconstruct paths.
- **Relaxation:** For an edge (u, v) with weight $w(u, v)$, the core operation is relaxation:

  $$\textbf{if } d[v] > d[u] + w(u, v) \textbf{ then } d[v] \leftarrow d[u] + w(u, v); \pi[v] \leftarrow u$$

  This updates the estimate for v if a shorter path is found via u.

Single-Source Shortest Paths (SSSP)

Given a graph $G = (V, E, w)$ and a source vertex s, find $\delta(s, v)$ for all $v \in V$.

BFS (Unweighted Graphs)

As discussed previously, Breadth-First Search finds shortest paths in terms of the *number of edges* (equivalent to all weights being 1). Time: $O(V + E)$.

Dijkstra's Algorithm (Non-negative Weights)

Problem: SSSP where all edge weights $w(e) \geq 0$.

Algorithm Overview: A greedy algorithm similar to Prim's MST. It maintains a set S of vertices for which the final shortest path distance from s has been determined. It repeatedly selects the vertex $u \in V \setminus S$ with the minimum current distance estimate $d[u]$, adds u to S, and relaxes all edges outgoing from u. A min-priority queue (PQ) stores vertices in $V \setminus S$, keyed by their distance estimates.

Detailed Steps:

1. **Initialize:**
   - For all $v \in V$, set $d[v] = \infty$, $\pi[v] = \text{NIL}$.
   - Set $d[s] = 0$.
   - Initialize set $S = \emptyset$.
   - Initialize min-priority queue Q containing all vertices V, keyed by d.
2. **Main Loop:** While Q is not empty:
   - Extract vertex $u = \texttt{Extract-Min}(Q)$.
   - Add u to S.
   - For each neighbor v of u ($(u, v) \in E$):
     - **Relax** (u, v)**:** If $d[v] > d[u] + w(u, v)$:
       * $d[v] = d[u] + w(u, v)$
       * $\pi[v] = u$
       * `Decrease-Key`(Q, v, d[v]) *(Update priority in PQ)*

Data Structures & Runtimes:

Implementation	PQ Data Structure	ExtractMin	DecreaseKey	Total Runtime	Bes
Naive	Array Scan	$O(V)$	$O(1)$	$\boldsymbol{O(V^2)}$	Der
Standard	Binary Heap	$O(\log V)$	$O(\log V)$	$\boldsymbol{O(E \log V)}$	Spa
Advanced	Fibonacci Heap	$O(\log V)^*$	$O(1)^*$	$\boldsymbol{O(E + V \log V)}$	Der

*Amortized cost

Proof of Correctness (Sketch): Uses induction on the size of the set S. The invariant is that for all $u \in S$, $d[u] = \delta(s, u)$.

- **Base Case:** $S = \{s\}$, $d[s] = 0 = \delta(s, s)$. Correct.

- **Inductive Step:** Assume $d[x] = \delta(s, x)$ for all $x \in S$. Let u be the next vertex added to S (the one with minimum $d[u]$ among $V \setminus S$). We need to show $d[u] = \delta(s, u)$.
- Consider any other path P' from s to u. Since $w(e) \geq 0$, path weights only increase. Let y be the first vertex on P' that is in $V \setminus S$ ($x = \pi[y] \in S$). Path $s \rightsquigarrow x \to y \rightsquigarrow u$.
- By inductive hypothesis $d[x] = \delta(s, x)$. When x was added to S, edge (x, y) was relaxed, so $d[y] \leq d[x] + w(x, y) \leq w(P'_{s \to y})$.
- Since u was chosen by Dijkstra instead of y, $d[u] \leq d[y]$.
- Combining: $d[u] \leq d[y] \leq w(P'_{s \to y}) \leq w(P'_{s \to u})$.
- Since $d[u]$ is the weight of a specific path found ($s \rightsquigarrow \pi[u] \to u$) and is less than or equal to the weight of any other path P', $d[u]$ must be the shortest path weight, $d[u] = \delta(s, u)$.

The non-negativity of weights is crucial because it ensures that once a vertex u is added to S, its distance $d[u]$ is final and won't be improved later by a path going through vertices still outside S.

Limitations: Dijkstra's fails if there are negative edge weights, as the greedy choice might be incorrect. A node added early to S might later be reachable via a much cheaper path involving a negative edge.

Applications & Reductions:

- **Distance to Hub:** Find shortest distance from any node to a set of hubs H. Add a dummy source s_0 with 0-weight edges to all $h \in H$. Run Dijkstra from s_0. The distance $d[v]$ is the shortest distance to any hub. Time: $O(E + V \log V)$.
- **Swim in Rising Water:** Find min time t to get from (0,0) to (N-1, N-1) in a grid where movement requires cell elevation $\leq t$. Model as a graph where cells are nodes. The "cost" of traversing an edge between adjacent cells u, v isn't additive; it's determined by the maximum elevation needed: $\text{cost}(u, v) = \max(\text{elev}(u), \text{elev}(v))$. The path cost is the maximum elevation encountered. Dijkstra can solve this by defining the distance $d[v]$ as the minimum time (max elevation) needed to reach cell v. Relaxation becomes: If $\max(d[u], \text{elev}(v)) < d[v]$, update $d[v]$.

Bellman-Ford Algorithm (Handles Negative Weights)

Problem: SSSP, works with negative edge weights. Detects reachable negative-weight cycles.

Algorithm Overview: Uses dynamic programming. It iteratively relaxes all edges in the graph. After k passes, it guarantees to find the shortest paths using at most k edges. Since simple shortest paths (no cycles) have at most $|V| - 1$ edges, $|V| - 1$ passes suffice if no negative cycles exist.

Detailed Steps:

1. **Initialize:**
   - For all $v \in V$, set $d[v] = \infty$, $\pi[v] = \text{NIL}$.
   - Set $d[s] = 0$.

2. **Relaxation Passes:** Repeat $|V| - 1$ times:
   - For each edge $(u, v) \in E$:
     - Relax (u, v). *(If $d[v] > d[u] + w(u, v)$, update $d[v], \pi[v]$)*
3. **Negative Cycle Check:** Repeat one more time (the $|V|$-th pass):
   - For each edge $(u, v) \in E$:
     - If $d[v] > d[u] + w(u, v)$:
       * A negative-weight cycle reachable from s exists. Report failure or handle appropriately.
4. **Return:** If no negative cycle detected, the distances $d[v]$ are the shortest path weights.

Sequential vs Parallel Updates:

- **Sequential (In-Place):** The pseudocode above uses sequential updates. When an edge (u, v) is relaxed, $d[v]$ is updated immediately and this new value might be used to relax another edge (v, x) within the same pass.
- **Parallel (Layered):** Maintains distances based on the number of edges. Let $d_k[v]$ be the shortest path using $\leq k$ edges. Then $d_k[v] = \min(d_{k-1}[v], \min_{(u,v)\in E}\{d_{k-1}[u] + w(u, v)\})$. Requires storing distances from the previous pass $(k - 1)$ to compute distances for the current pass (k).

Proof of Correctness (Sketch): Let $\delta_k(s, v)$ be the true shortest path weight from s to v using at most k edges.

- **Claim 1:** $d[v] \geq \delta(s, v)$ always holds (distances are upper bounds).
- **Claim 2:** After pass k, $d[v] \leq \delta_k(s, v)$. Proven by induction on k.
- Since any simple shortest path has at most $|V| - 1$ edges, after $|V| - 1$ passes, $d[v] \leq \delta_{|V|-1}(s, v) = \delta(s, v)$ (if no negative cycles).
- **Negative Cycles:** If distances can still be improved in pass $|V|$, it means some $d[v]$ is still decreasing. This can only happen if there's a path from s to v that includes a negative cycle, allowing the path weight to decrease indefinitely.

Runtime Analysis:

- $|V| - 1$ passes (or $|V|$ passes including check).
- Each pass iterates through all $|E|$ edges, performing $O(1)$ work per edge (relaxation).
- **Total Time:** $O(|V| \cdot |E|)$.

Applications & Reductions:

- **Cheapest Flights with K Stops:** Find min cost from 'src' to 'dst' using at most 'K' stops (i.e., at most 'K+1' flights/edges). Run the parallel-update version of Bellman-Ford for exactly $K + 1$ iterations. The final $d_{K+1}[\text{dst}]$ gives the answer. Time: $O(K \cdot |E|)$.

Shortest Paths in Directed Acyclic Graphs (DAGs)

Advantage: SSSP in DAGs can be solved faster than general graphs, even with negative edge weights ($O(V + E)$). DAGs inherently have no cycles, including negative ones.

Algorithm:

1. Topologically sort the vertices of the DAG.
2. Initialize distances: $d[s] = 0$, $d[v] = \infty$ for $v \neq s$.
3. Process vertices u in topological order:
   - For each outgoing edge $(u, v) \in E$:
     - Relax (u, v). *(If $d[v] > d[u] + w(u, v)$, update $d[v], \pi[v]$)*

Correctness: When vertex u is processed, due to the topological order, all vertices x with an edge (x, u) must have already been processed. This ensures that the current value $d[u]$ represents the true shortest path distance $\delta(s, u)$. Therefore, relaxing outgoing edges (u, v) correctly propagates the shortest path information.

Runtime Analysis:

- Topological Sort: $O(V + E)$.
- Initialization: $O(V)$.
- Relaxation: Each edge is relaxed exactly once: $O(E)$.
- **Total Time:** $O(V + E)$.

All-Pairs Shortest Paths (APSP)

Find the shortest path weight $\delta(u, v)$ for all pairs $(u, v) \in V \times V$.

Floyd-Warshall Algorithm

Problem: APSP, handles negative weights (if no negative cycles).

Algorithm Overview: Dynamic programming. Iteratively considers each vertex k as a potential intermediate vertex on the shortest path between any pair (i, j).

DP Formulation: Let $d_k[i][j]$ be the shortest path weight from i to j using only intermediate vertices from $\{1, 2, \ldots, k\}$.

$$d_k[i][j] = \min(d_{k-1}[i][j], \quad d_{k-1}[i][k] + d_{k-1}[k][j])$$

The shortest path either uses k or it doesn't.

Detailed Steps (In-place):

1. Initialize $n \times n$ matrix D:
   - $D[i][i] = 0$.
   - $D[i][j] = w(i, j)$ if edge (i, j) exists.
   - $D[i][j] = \infty$ if $i \neq j$ and no direct edge exists.

2. (Optional) Initialize predecessor matrix $P[i][j] = i$ if edge (i, j) exists or $i = j$, else NIL.

3. For k from 1 to n (*intermediate vertex*):

   - For i from 1 to n (*source vertex*):
     - For j from 1 to n (*destination vertex*):
       * If $D[i][k] + D[k][j] < D[i][j]$:
         · $D[i][j] = D[i][k] + D[k][j]$.
         · (Optional) $P[i][j] = P[k][j]$.

4. **Negative Cycle Check:** After loops, check if $D[i][i] < 0$ for any i. If so, a negative cycle exists.

5. Return D.

Path Reconstruction: Using matrix P, recursively find path from i to $P[i][j]$ and path from $P[i][j]$ to j.

Runtime Analysis:

- Three nested loops: $O(V^3)$.
- Initialization: $O(V^2)$.
- **Total Time:** $O(V^3)$.
- **Space Complexity:** $O(V^2)$ for distance matrix (and optional predecessor matrix).

Applications & Reductions:

- **City with Fewest Neighbors within Threshold:** Compute all-pairs distances using Floyd-Warshall. Then, for each city i, iterate through all j and count how many satisfy $D[i][j] \leq$ threshold. Find the city with the minimum count (breaking ties by city ID). Time: $O(V^3 + V^2) = O(V^3)$.

Johnson's Algorithm (Sparse Graphs with Negative Weights)

Goal: Achieve better performance than Floyd-Warshall on sparse graphs ($|E| \ll |V|^2$), while still handling negative weights (but no negative cycles).

Algorithm Overview: Uses Bellman-Ford for reweighting and then runs Dijkstra from each node.

Detailed Steps:

1. **Reweighting Setup:**
   - Create $G' = (V', E')$ where $V' = V \cup \{s_0\}$ (new source s_0).
   - $E' = E \cup \{(s_0, v) \mid v \in V\}$.
   - Set $w(s_0, v) = 0$ for all $v \in V$. Original edge weights remain.

2. **Bellman-Ford Pass:**
   - Run Bellman-Ford on G' starting from s_0.

- If a negative cycle is detected, report it and terminate.
- Otherwise, let $h(v) = \delta(s_0, v)$ be the shortest path weight from s_0 found by Bellman-Ford.

3. **Reweight Edges:** For each original edge $(u, v) \in E$:
   - Calculate new weight $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$.
   - By the triangle inequality property of shortest paths, $\hat{w}(u, v) \geq 0$.
4. **Dijkstra Passes:**
   - For each vertex $u \in V$:
     - Run Dijkstra's algorithm on G starting from u, using the non-negative weights $\hat{w}$.
     - Let $\hat{\delta}(u, v)$ be the shortest path weight found by Dijkstra from u to v using weights $\hat{w}$.
5. **Compute Final Distances:** For each pair (u, v):
   - The original shortest path weight is $\delta(u, v) = \hat{\delta}(u, v) - h(u) + h(v)$.

Correctness: The reweighting step ensures all edge weights are non-negative while preserving the shortest path structure (due to the telescoping property of the potentials $h(v)$). Dijkstra can then be correctly applied.

Runtime Analysis:

- Bellman-Ford: $O(VE)$.
- $|V|$ runs of Dijkstra:
  - With binary heap: $|V| \times O(E \log V) = O(VE \log V)$.
  - With Fibonacci heap: $|V| \times O(E + V \log V) = O(VE + V^2 \log V)$.
- Reweighting and final conversion: $O(E + V^2)$.
- **Total Time (Fibonacci Heap):** $O(VE + V^2 \log V)$. This is better than $O(V^3)$ for sparse graphs where E is significantly smaller than V^2.

Network Flow

Network flow problems model scenarios where some commodity (data, goods, etc.) moves through a network with limitations (capacities). The primary goal is often to maximize the amount of flow from a source to a destination (sink).

Problem Definition: Maximum Flow

- **Input:** A directed graph $G = (V, E)$, a source vertex $s \in V$, a sink vertex $t \in V$, and a non-negative capacity $c(u, v) \geq 0$ for each edge $(u, v) \in E$. (If $(u, v) \notin E$, assume $c(u, v) = 0$).
- **Flow:** A function $f : V \times V \to \mathbb{R}$ satisfying:
  - **Capacity Constraint:** For all $u, v \in V$, $f(u, v) \leq c(u, v)$. Flow cannot exceed capacity.
  - **Skew Symmetry:** For all $u, v \in V$, $f(u, v) = -f(v, u)$. (Conceptual: flow from u to v is negative flow from v to u). Often, we only consider non-negative flow on existing edges.
  - **Flow Conservation:** For all vertices $u \in V \setminus \{s, t\}$, the total flow entering u equals the total flow leaving u:

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) \quad \text{or equivalently} \quad \sum_{v \in V} f(u, v) = 0$$

- **Value of Flow $|f|$:** The total net flow out of the source s:

$$|f| = \sum_{v \in V} f(s, v)$$

  (Due to conservation, this also equals the net flow into the sink t).
- **Goal:** Find a flow f such that $|f|$ is maximized.

(Illustrative images like ‘maxflow1.png‘, ‘maxflow2.png‘, ‘maxflow3.png‘ show examples of capacities, partial flow, and maximum flow)

The Ford-Fulkerson Method

A general method for finding the maximum flow. It iteratively increases the flow value along paths from s to t.

Greedy Idea and Residual Graphs

A simple greedy approach might be: find any path from s to t with available capacity, push as much flow as possible along it (limited by the path’s bottleneck capacity), and repeat.

Problem: This simple greedy approach can get stuck in a suboptimal flow if a poor path choice is made early on (as shown in slide examples ‘maxflow-0001.png‘ to ‘maxflow-0004.png‘).

Solution: Residual Graphs and Backward Edges To allow "undoing" bad flow decisions, we introduce the concept of a residual graph G_f for a given flow f.

- **Vertices:** Same as G.
- **Edges & Residual Capacity c_f:** For each edge $(u, v) \in E$ in the original graph:

- **Forward Edge:** If $f(u,v) < c(u,v)$, include edge (u,v) in G_f with residual capacity $c_f(u,v) = c(u,v) - f(u,v)$. (Represents remaining capacity to push more flow).
- **Backward Edge:** If $f(u,v) > 0$, include edge (v,u) in G_f with residual capacity $c_f(v,u) = f(u,v)$. (Represents capacity to "push back" or cancel existing flow).

Augmenting Paths

- An **augmenting path** p is a simple path from s to t in the residual graph G_f.
- The **residual capacity** (or **bottleneck capacity**) of path p is $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ is on path } p\}$. This is the maximum amount of additional net flow that can be pushed along p.

Augmenting the Flow

Given an augmenting path p with capacity $b = c_f(p)$:

- For each edge (u,v) on p:
  - If (u,v) is a forward edge in G_f (corresponding to $(u,v) \in E$), increase $f(u,v)$ by b.
  - If (u,v) is a backward edge in G_f (corresponding to $(v,u) \in E$), decrease $f(v,u)$ by b.

This process increases the total flow value $|f|$ by exactly b.

Ford-Fulkerson Algorithm (Generic)

1. Initialize flow $f(u,v) = 0$ for all $u, v \in V$.
2. While there exists an augmenting path p from s to t in the residual graph G_f:
   - Find such a path p (e.g., using DFS or BFS on G_f).
   - Calculate its residual capacity $b = c_f(p)$.
   - Augment the flow f by b along path p.
   - Update the residual graph G_f based on the new flow f.
3. Return the final flow f.

Properties and Analysis

- **Integrality Theorem:** If all capacities $c(u,v)$ are integers, Ford-Fulkerson (if it terminates) produces a maximum flow f where all flow values $f(u,v)$ are also integers.
- **Termination:** If capacities are integers, the flow increases by at least 1 in each augmentation. Since the maximum flow value $|f^*|$ is bounded (e.g., by the sum of capacities out of s), the algorithm terminates in at most $|f^*|$ augmentations.

- **Runtime (Generic):** Finding an augmenting path takes $O(E)$ (using BFS/DFS). Total time: $O(E \cdot |f^*|)$. This can be slow if $|f^*|$ is large.
- **Non-termination:** If capacities are irrational, Ford-Fulkerson might not terminate.

Cuts and the Max-Flow Min-Cut Theorem

How do we know when the flow is maximum? The concept of cuts provides the answer.

s-t Cuts

- An **s-t cut** (S, T) is a partition of the vertex set V into two disjoint sets S and $T = V \setminus S$ such that $s \in S$ and $t \in T$.
- The **capacity** of the cut is the sum of capacities of edges going from S to T:

$$\text{cap}(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

  (Note: only considers edges in the original direction).
- The **net flow** across the cut is the total flow out of S minus the total flow into S:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in T} \sum_{v \in S} f(v, u)$$

Relationship between Flow and Cut

- **Property 1:** For any flow f and any s-t cut (S, T), the net flow across the cut equals the value of the flow: $f(S, T) = |f|$. (This follows from flow conservation for nodes in $S \setminus \{s\}$).
- **Property 2 (Weak Duality):** For any flow f and any s-t cut (S, T), the value of the flow is less than or equal to the capacity of the cut: $|f| \leq \text{cap}(S, T)$. *Proof Idea:* $|f| = f(S, T) = \sum_{S \to T} f(u, v) - \sum_{T \to S} f(v, u) \leq \sum_{S \to T} f(u, v) \leq \sum_{S \to T} c(u, v) = \text{cap}(S, T)$.
- This implies the maximum flow value is bounded by the minimum cut capacity.

Max-Flow Min-Cut Theorem

Theorem: In any flow network, the maximum value of an s-t flow is equal to the minimum capacity of an s-t cut.

Proof Idea (using Ford-Fulkerson):

1. Let f be the final flow when Ford-Fulkerson terminates.
2. Since no more augmenting paths exist, there is no path from s to t in the residual graph G_f.
3. Let S be the set of vertices reachable from s in G_f, and let $T = V \setminus S$.
4. Since $s \in S$ and $t \notin S$ (as t is unreachable), (S, T) is an s-t cut.

5. Consider edges between S and T in the original graph G:
   - For any edge (u, v) with $u \in S, v \in T$: We must have $c_f(u, v) = 0$ in G_f (otherwise v would be reachable from s via u). This means $f(u, v) = c(u, v)$. **(Forward edges are saturated)**.
   - For any edge (v, u) with $v \in T, u \in S$: We must have $c_f(u, v) = 0$ in G_f. But $c_f(u, v) = f(v, u)$. This means $f(v, u) = 0$. **(Backward edges are empty)**.
6. Now calculate the net flow across the cut (S, T):

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v) - \sum_{v \in T, u \in S} f(v, u)$$

$$f(S, T) = \sum_{u \in S, v \in T} c(u, v) - \sum_{v \in T, u \in S} 0 = \text{cap}(S, T)$$

7. Since $|f| = f(S, T)$, we have $|f| = \text{cap}(S, T)$.
8. We know $|f| \leq$ min cut capacity and we found a specific cut where $|f| =$ capacity. Therefore, $|f|$ must be the maximum flow, and $\text{cap}(S, T)$ must be the minimum cut capacity.

Edmonds-Karp Algorithm

A specific implementation of Ford-Fulkerson that chooses the augmenting path using **BFS** on the residual graph G_f. This guarantees finding the *shortest* augmenting path in terms of the number of edges.

Algorithm: Same as Ford-Fulkerson, but step 2a ("Find an augmenting path p") is specifically done using BFS.

Runtime Analysis:

- BFS takes $O(E)$ time.
- It can be proven that the length of the shortest augmenting path found by BFS is non-decreasing over iterations.
- An edge can become a bottleneck (critical edge on the shortest path) at most $|V|/2$ times.
- The total number of augmentations is bounded by $O(VE)$.
- **Total Time:** $O(VE^2)$. This bound is independent of the maximum flow value and guarantees polynomial time even for large capacities.

Applications: Reductions to Max Flow

Max flow is a powerful tool applicable to problems involving matching, resource allocation, scheduling, and network connectivity.

Edge-Disjoint Paths

Problem: Find the maximum number of paths from s to t that do not share any edges. **Reduction:**

1. Assign capacity $c(e) = 1$ to every edge $e \in E$.
2. Compute the maximum s-t flow value $|f^*|$ in this unit-capacity network.

Result: The maximum number of edge-disjoint paths is equal to $|f^*|$. **Proof Idea:** An integer flow (guaranteed by Integrality Theorem) corresponds to selecting paths. Capacity 1 ensures edges are used at most once across all paths. **Menger's Theorem (Edge Version):** The max number of edge-disjoint s-t paths equals the min number of edges whose removal disconnects t from s (min edge cut). This follows from Max-Flow Min-Cut in the unit-capacity graph.

Vertex-Disjoint Paths

Problem: Find the maximum number of paths from s to t that do not share any intermediate vertices (they can share s and t). **Reduction (Vertex Splitting):**

1. Create a new graph G'.
2. For each vertex $v \in V \setminus \{s, t\}$, replace it with two vertices v_{in} and v_{out}, and add a directed edge (v_{in}, v_{out}) with capacity 1 (this enforces the vertex constraint). For s and t, consider $s = s_{in} = s_{out}$ and $t = t_{in} = t_{out}$ conceptually, or handle them separately. Alternatively, split s and t too but give (s_{in}, s_{out}) and (t_{in}, t_{out}) infinite capacity. The standard approach splits only intermediate vertices.
3. For each original edge $(u, v) \in E$, add an edge (u_{out}, v_{in}) in G' with infinite capacity (or capacity 1 if original edge capacity matters beyond just path counting).
4. Assign capacity 1 to the internal edges (v_{in}, v_{out}).
5. Compute the maximum flow from s_{out} (or s) to t_{in} (or t) in G'.

Result: The max flow value equals the maximum number of vertex-disjoint paths. Flow through (v_{in}, v_{out}) ensures vertex v is used at most once across all paths. **Menger's Theorem (Vertex Version):** Max number of vertex-disjoint s-t paths equals min number of vertices whose removal disconnects t from s.

Maximum Bipartite Matching

Problem: Given a bipartite graph $G = (L \cup R, E)$, find a matching $M \subseteq E$ (no two edges share a vertex) of maximum size $|M|$. **Reduction:**

1. Create a flow network G'.
2. Add source s and sink t.
3. Add edges (s, u) for all $u \in L$ with capacity 1.
4. Add edges (v, t) for all $v \in R$ with capacity 1.

5. For each edge $(u, v) \in E$ (with $u \in L, v \in R$), add a directed edge (u, v) in G' with capacity 1 (or ∞).
6. Compute the max flow from s to t in G'.

Result: The max flow value equals the size of the maximum matching. Edges (u, v) between L and R carrying flow correspond to edges in the matching. **Hall's Marriage Theorem Connection:** Hall's condition ($|N(S)| \geq |S|$ for all $S \subseteq L$) is necessary and sufficient for a perfect matching (size $|L|$). This can be proven using the Max-Flow Min-Cut theorem on the constructed network, showing the min cut capacity is $|L|$ iff Hall's condition holds.

Example: Exam Scheduling

Problem: Schedule c courses in r rooms over t time slots using p proctors, subject to constraints (enrollment $\leq$ room capacity, room/time slot uniqueness, proctor availability, proctor load limit). **Reduction:** Construct a multi-layered flow network:

- Nodes: s, t, Course nodes C_i, Room nodes R_j, TimeSlot nodes T_l, Proctor nodes P_k.
- Edges & Capacities: * $s \to C_i$: cap 1 (Each course scheduled once) * $C_i \to R_j$: cap 1 (if $E_i \leq S_j$) * $R_j \to T_l$: cap 1 (Room j used once at time l) * $T_l \to P_k$: cap 1 (if proctor k available at time l) * $P_k \to t$: cap 3 (Proctor k supervises at most 3 exams)

Result: Compute max flow $|f^*|$ from s to t.

- If $|f^*| = c$ (number of courses), a valid schedule exists.
- Each unit of flow along a path $s \to C_i \to R_j \to T_l \to P_k \to t$ corresponds to scheduling course i in room j at time l with proctor k.
- If $|f^*| < c$, no valid schedule exists.

Correctness: The capacities enforce all problem constraints. Max flow finds the maximum number of possible valid assignments.

Computational Complexity

Computational complexity theory analyzes the resources (primarily time and space) required to solve computational problems. It helps classify problems based on their inherent difficulty and understand the limits of efficient computation.

Models of Computation and Efficiency

- **Model of Computation:** To analyze resources formally, we need a model of a computer. While the **Turing Machine** is the standard theoretical model, for algorithm analysis we often use the **Random Access Machine (RAM) model**, which assumes basic operations (arithmetic, comparison, memory access) take constant time. Other models exist for specific contexts (circuits, parallel machines, streaming, quantum).

- **Input Size:** Complexity is measured as a function of the input size N, typically the number of bits needed to represent the input instance.
- **Polynomial Time:** An algorithm runs in polynomial time if its runtime is bounded by $O(N^k)$ for some constant $k \geq 0$. Such algorithms are generally considered **efficient** or **tractable**. Examples include Merge Sort ($O(N \log N)$), Dijkstra's ($O(E \log V)$), and many dynamic programming solutions where parameters are polynomial in N.
- **Pseudo-Polynomial Time:** Algorithms whose runtime is polynomial in the numeric value of some input numbers, but exponential in the length (number of bits) of those numbers. Examples: Knapsack ($O(nW)$), Integer Subset Sum ($O(nB)$). These are not truly polynomial if W or B can grow exponentially large relative to the input length N.

Decision Problems and Verification

Complexity theory often focuses on **decision problems**, which have a YES/NO (or TRUE/FALSE, 1/0) answer.

- **Examples:** Is graph G connected? Does graph G have a 3-coloring? Does formula Φ have a satisfying assignment?
- Optimization or search problems ("Find the best...") can often be converted to decision problems ("Is there a solution with value at least/at most K?").
- **Verifier:** An algorithm A is a verifier for a decision problem Π if, given an instance x and a potential proof or "certificate" y, $A(x, y)$ outputs TRUE if y is a valid proof that the answer to $\Pi(x)$ is YES, and FALSE otherwise.
- **Polynomial-Time Verification:** A problem is poly-time verifiable if there exists a verifier A that runs in time polynomial in the size of the instance $|x|$ (often requires the certificate $|y|$ to also be polynomial in $|x|$).
- **Example (Hamiltonian Cycle):**
  - **Problem:** Does graph G contain a Hamiltonian cycle?
  - **Certificate y:** A sequence of vertices $(v_1, v_2, \ldots, v_n)$.
  - **Verifier $A(G, y)$:** Checks in poly-time if y forms a simple cycle visiting all vertices in G.
- **Asymmetry:** Verification usually applies only to YES instances. Proving a NO answer (e.g., proving a graph has no Hamiltonian cycle) often seems much harder and may not have a short, easily verifiable certificate.

Complexity Classes P and NP

- **Class P (Polynomial Time):** The set of all decision problems that can be **solved** by a deterministic algorithm in polynomial time $O(|x|^k)$. These are considered efficiently solvable problems. *Examples:* Connectivity, 2-SAT, MST weight $\leq K$, Shortest Path length $\leq K$.

- **Class NP (Nondeterministic Polynomial Time):** The set of all decision problems for which a YES answer can be **verified** by a deterministic algorithm in polynomial time, given a suitable certificate (proof). *Examples:* Hamiltonian Cycle, 3-Coloring, 3-SAT, Clique size $\geq K$, Vertex Cover size $\leq K$, Subset Sum.
- **Relationship:** $P \subseteq NP$. If a problem can be solved in poly-time, the solution itself (or the process) can serve as a certificate, and verification involves just re-running the solver.
- **The P vs NP Question:** Is $P = NP$? This is the most famous open question in theoretical computer science. Does efficient verification imply efficient solvability? Most researchers believe $P \neq NP$, implying that there are problems in NP that cannot be solved efficiently by any deterministic algorithm.

NP-Hardness and NP-Completeness

To understand the structure within NP and the "hardest" problems, we use reductions.

- **Polynomial-Time Reduction ($A \leq_P B$):** Problem A is poly-time reducible to problem B if there exists a poly-time algorithm (the reduction) that transforms any instance x of A into an instance $f(x)$ of B, such that the answer to $A(x)$ is YES if and only if the answer to $B(f(x))$ is YES. *Meaning:* If we have a "black box" poly-time solver for B, we can solve A in poly-time. Thus, B is at least as hard as A.
- **NP-Hard:** A problem H is NP-hard if *every* problem $A \in NP$ is poly-time reducible to H ($A \leq_P H$ for all $A \in NP$). NP-hard problems are at least as hard as any problem in NP. They do not necessarily have to be decision problems or be in NP themselves (e.g., TSP optimization version, Halting Problem). If any NP-hard problem can be solved in poly-time, then $P = NP$.
- **NP-Complete (NPC):** A problem C is NP-complete if: 1. $C \in NP$ (It is efficiently verifiable). 2. C is NP-hard (All other NP problems reduce to it). NPC problems are the "hardest" problems within NP.
- **Cook-Levin Theorem (1971):** Proved that the Boolean Satisfiability problem (SAT), and specifically Circuit-SAT, is NP-complete. This provided the first "anchor" NPC problem. Later shown that 3-SAT is also NPC.

Proving NP-Completeness via Reductions

To prove a decision problem Y is NP-complete:

1. **Show $Y \in NP$:** Design a polynomial-time verifier for Y. Describe the certificate and how the verifier uses it to check a YES instance in polynomial time.
2. **Show Y is NP-hard:**
   - Choose a known NP-complete problem X (e.g., 3-SAT, Vertex Cover, Clique).
   - Construct a polynomial-time reduction f mapping instances x of X to instances $y = f(x)$ of Y.

- Prove the reduction's correctness: Show that x is a YES instance of X **if and only if** $y = f(x)$ is a YES instance of Y. This typically involves proving both directions ($\Rightarrow$ and $\Leftarrow$).

Examples of NP-Completeness Reductions

Circuit-SAT $\leq_P$ 3-SAT

Goal: Show 3-SAT is NP-hard by reducing Circuit-SAT to it.

Reduction: Given a boolean circuit C:

1. Introduce a boolean variable for each wire in C (inputs, outputs, intermediates).
2. For each gate, add clauses to enforce its logical behavior:
   - NOT gate ($a = \neg b$): Add $(a \vee b) \wedge (\neg a \vee \neg b)$.
   - OR gate ($a = b \vee c$): Add $(\neg a \vee b \vee c) \wedge (a \vee \neg b) \wedge (a \vee \neg c)$. (This is already 3-CNF).
   - AND gate ($a = b \wedge c$): Add $(a \vee \neg b \vee \neg c) \wedge (\neg a \vee b) \wedge (\neg a \vee c)$. (This is 3-CNF).
3. Add a clause consisting of just the output wire variable (z) to ensure the circuit output must be 1.

Let the resulting conjunction of all these clauses be Φ.

Correctness:

- ($\Rightarrow$) If C has a satisfying input x, assign the corresponding wire values in Φ. Each gate clause is satisfied by construction, and the output clause (z) is satisfied since $C(x) = 1$. Thus Φ is satisfiable.
- ($\Leftarrow$) If Φ has a satisfying assignment y, the values assigned to the input wire variables form an input x for C. The gate clauses ensure all wire values correspond to correct gate operations. The output clause (z) ensures $C(x) = 1$. Thus C is satisfiable.

Runtime: The number of variables and clauses in Φ is linear in the size (number of wires and gates) of C. The reduction is polynomial time.

Conclusion: Since Circuit-SAT is NP-complete and Circuit-SAT $\leq_P$ 3-SAT, 3-SAT is NP-hard. Since 3-SAT is also in NP, **3-SAT is NP-complete**.

3-SAT $\leq_P$ Maximum Independent Set (MIS)

Goal: Show MIS (decision version: Is there an independent set of size $\geq k$?) is NP-hard. We reduce 3-SAT to MIS.

Reduction: Given a 3-CNF formula Φ with m clauses $C_1, \ldots, C_m$. Construct graph $G = (V, E)$:

1. **Vertices:** For each clause $C_r = (l_{r1} \vee l_{r2} \vee l_{r3})$, create three vertices v_{r1}, v_{r2}, v_{r3}, one for each literal. $|V| = 3m$.
2. **Edges:**

- Add edges between all pairs of vertices corresponding to literals within the same clause (forms a triangle for each clause). Ensures at most one vertex per clause triangle can be in an independent set.
- Add an edge between any two vertices v_{ri} and v_{sj} if their corresponding literals are negations of each other (e.g., $l_{ri} = x$ and $l_{sj} = \neg x$). Ensures complementary literals cannot both be chosen.

3. Set the target independent set size $k = m$.

Correctness: Φ is satisfiable $\iff$ G has an independent set of size m.

- ($\Rightarrow$) If Φ is satisfiable by assignment A, then each clause C_r has at least one true literal. For each clause C_r, choose one vertex v_{ri} corresponding to a true literal l_{ri} under A. Let S be the set of these m chosen vertices.
  - S is independent: No two vertices in S are from the same clause triangle (we chose one per clause). No two vertices correspond to complementary literals (if x is true, $\neg x$ is false, so we wouldn't choose the vertex for $\neg x$).
  - $|S| = m$.
- ($\Leftarrow$) If G has an independent set S of size $k = m$.
  - Since vertices in each clause form a triangle, S must contain exactly one vertex from each clause triangle.
  - Define a truth assignment A: If $v_{ri} \in S$ corresponds to literal x, set $x =$ TRUE. If $v_{ri} \in S$ corresponds to literal $\neg x$, set $x =$ FALSE. Assign arbitrary values to variables not represented in S.
  - This assignment is consistent because S cannot contain vertices for both x and $\neg x$ (due to edges between them).
  - This assignment satisfies Φ because each clause C_r contributes one vertex to S, and the literal corresponding to that vertex must be true under A.

Runtime: Construction takes polynomial time.

Conclusion: MIS is NP-hard. Since MIS is also in NP (certificate: the set S; verifier: check size and independence in poly-time), **MIS is NP-complete**.

Maximum Independent Set (MaxIndSet) $\leq_P$ Minimum Vertex Cover (MinVC)

Goal: Show MinVC (decision: Is there a vertex cover of size $\leq k$?) is NP-hard.

Key Relationship: In any graph $G = (V, E)$, a set $S \subseteq V$ is an independent set if and only if its complement $T = V \setminus S$ is a vertex cover.

- (S is IS $\Rightarrow$ $V \setminus S$ is VC): If $T = V \setminus S$ is not a VC, there exists an edge (u, v) with both $u, v \notin T$. This means $u, v \in S$. But S is independent, so no edge exists between u, v. Contradiction.
- (T is VC $\Rightarrow$ $V \setminus T$ is IS): If $S = V \setminus T$ is not independent, there exists an edge (u, v) with $u, v \in S$. This means $u, v \notin T$. But T is a vertex cover, so every edge must have at least one endpoint in T. Contradiction.

Size Relationship: $|S| + |T| = |V|$. Maximizing $|S|$ is equivalent to minimizing $|T|$.

Reduction: Given an instance (G, k) of MaxIndSet (Does G have an IS of size $\geq k$?):

1. Construct an instance (G, k') of MinVC where $k' = |V| - k$.
2. Ask: Does G have a vertex cover of size $\leq k'$?

Correctness: G has an IS of size $\geq k \iff G$ has a VC of size $\leq |V| - k = k'$.

Conclusion: MinVC is NP-hard. Since MinVC is also in NP (certificate: the set T; verifier: check size and cover property), **MinVC is NP-complete**.

(Maximum Clique is also NPC, reducible from MaxIndSet by considering the complement graph $\bar{G}$: S is an IS in $G \iff S$ is a Clique in $\bar{G}$).

3-Coloring $\leq_P$ Relaxed 3-Coloring

Problem (Relaxed 3-Coloring): Given G, can its vertices be 3-colored such that at most $k = 31337$ edges have both endpoints the same color?

Goal: Show Relaxed 3-Coloring is NP-hard by reducing standard 3-Coloring (which is NPC) to it.

Reduction:

1. Given an instance $G = (V, E)$ of 3-Coloring.
2. Construct G': Start with G. Add $k = 31337$ disjoint copies of K_4 (a complete graph on 4 vertices).
3. Ask: Is G' 3-colorable with at most k monochromatic edges?

Correctness:

- K_4 requires 4 colors for a proper coloring. Any 3-coloring of K_4 *must* have at least one monochromatic edge. It's possible to 3-color K_4 with exactly one monochromatic edge.
- ($\Rightarrow$) If G is 3-colorable (0 monochromatic edges), then color G properly. Color each K_4 copy using 3 colors such that it has exactly one monochromatic edge. The total number of monochromatic edges in G' is exactly k. So, the answer to Relaxed 3-Coloring on G' is YES.
- ($\Leftarrow$) If G' has a 3-coloring with at most k monochromatic edges. Since each of the k copies of K_4 must contribute at least one monochromatic edge, they must contribute exactly k such edges in total. This means the original subgraph G must have 0 monochromatic edges in this coloring. Thus, G is 3-colorable.

Runtime: Adding k copies of K_4 takes polynomial time (as k is a constant).

Conclusion: Relaxed 3-Coloring is NP-hard. (It's also in NP, thus NPC).

Hamiltonian Path $\leq_P$ Bounded-Degree Spanning Tree

Problem (Bounded-Degree Spanning Tree - BDST): Given $G = (V, E)$, does G contain a spanning tree where every vertex has degree at most $D = 7$?

Goal: Show BDST (with $D = 7$) is NP-hard by reducing Undirected s-t Hamiltonian Path (which is NPC) to it.

Reduction: Given an instance $(G = (V, E), s, t)$ of s-t HamPath:

1. Construct graph G': Start with G.
2. For each $v \in V \setminus \{s, t\}$, add 5 new "dummy" vertices $d_{v1}, \ldots, d_{v5}$ and edges (v, d_{vi}) for $i = 1..5$.
3. For s, add 6 dummy vertices $d_{s1}, \ldots, d_{s6}$ and edges (s, d_{si}).
4. For t, add 6 dummy vertices $d_{t1}, \ldots, d_{t6}$ and edges (t, d_{ti}).
5. Ask: Does G' have a spanning tree with maximum degree at most $D = 7$?

Correctness:

- ($\Rightarrow$) If G has an s-t HamPath P. Construct a tree T' in G' consisting of the path P plus all edges connecting original vertices to their corresponding dummy vertices.
  - T' is spanning and connected. It's a tree because P is acyclic and dummies attach only to P.
  - Degrees in T': * Dummy vertex: degree 1. * $v \in V \setminus \{s, t\}$: degree in P is 2. Degree in T' is $2 + 5 = 7$. * s: degree in P is 1. Degree in T' is $1 + 6 = 7$. * t: degree in P is 1. Degree in T' is $1 + 6 = 7$.
  - All degrees are ≤ 7. So the answer to BDST on G' is YES.
- ($\Leftarrow$) If G' has a spanning tree T' with max degree ≤ 7.
  - Each dummy vertex must be connected to its corresponding original vertex in T' (it's the only possible connection).
  - Consider the degrees of original vertices in T'. Vertex $v \in V \setminus \{s, t\}$ has degree ≤ 7. Since 5 edges go to its dummies, its degree using original edges from E must be $\leq 7 - 5 = 2$.
  - Vertex s has degree ≤ 7. Since 6 edges go to its dummies, its degree using original edges must be $\leq 7 - 6 = 1$. Similarly for t.
  - Let T be the subgraph of T' restricted to original vertices V and original edges E. T must be connected (since T' was connected). The degrees in T are ≤ 2 for internal vertices and ≤ 1 for s, t. The only connected graph with n vertices satisfying these degree constraints and having $n - 1$ edges (since it must be a tree or forest derived from T') is a simple path starting at s and ending at t. This path must include all vertices (otherwise T' wasn't spanning). Thus, it's an s-t Hamiltonian path. The answer to HamPath on G is YES.

Runtime: Adding dummies takes polynomial time.

Conclusion: BDST (with D=7) is NP-hard. (It is also in NP, hence NPC).

Maximum Independent Set $\leq_P$ Maximum Pyramid

Problem (Maximum Pyramid): Given $G = (V, E)$, find the size of the largest "pyramid". A pyramid is a set $\{v_1, \ldots, v_k\} \cup \{u\}$ such that $\{v_1, \ldots, v_k\}$ is an independent set, and u is adjacent to all v_i. The size is $k + 1$.

Goal: Show Max Pyramid is NP-hard by reducing Max Independent Set to it.

Reduction: Given an instance $G = (V, E)$ of MaxIndSet:

1. Construct graph G': Start with G. Add a new vertex z. Add edges (z, v) for all $v \in V$.
2. Find the size K of the largest pyramid in G'.
3. Output $K - 1$ as the size of the largest independent set in G.

Correctness: Max pyramid size in G' = (Max IS size in G) + 1.

- ($\Rightarrow$) If G has an IS $S = \{v_1, \ldots, v_k\}$. Then $S \cup \{z\}$ forms a pyramid in G' of size $k+1$, because S is independent and z is connected to all v_i. So, MaxPyramidSize(G') $\geq$ MaxISSize(G) + 1.
- ($\Leftarrow$) If G' has a pyramid $P = \{v_1, \ldots, v_k\} \cup \{u\}$ of size $k+1$. The set $S = \{v_1, \ldots, v_k\}$ must be an independent set in G'. Since z is connected to all other vertices, none of the v_i can be z. So, S is a subset of the original vertices V. Since edges within V are the same in G and G', S is an independent set in G of size k. Thus, MaxISSize(G) $\geq$ MaxPyramidSize(G') - 1.

Combining these gives equality.

Runtime: Adding z and its edges takes polynomial time.

Conclusion: Maximum Pyramid is NP-hard. (Finding the size is an optimization problem; the corresponding decision problem "Does G have a pyramid of size at least K?" is NPC).

3-Coloring $\leq_P$ Relaxed 3-Coloring ($\leq k$ bad edges)

Problem (Relaxed 3-Coloring): Given an undirected graph G, is it possible to color the vertices of G with three different colors, so that at most $k = 31337$ edges have both endpoints the same color?

Goal: Show Relaxed 3-Coloring is NP-hard by reducing standard 3-Coloring (which is NPC) to it.

Reduction from 3-coloring:

1. Given an instance $G = (V, E)$ of the standard 3-coloring problem.
2. Create an instance G' of the relaxed 3-coloring problem as follows: Start with G. Add $k = 31337$ disjoint copies of K_4 (a complete graph on 4 vertices) to G. Let G' be the resulting graph.
3. Ask: Is G' 3-colorable with at most k monochromatic edges?

Proof of correctness: We show that G is 3-colorable if and only if G' is 3-colorable with at most $k = 31337$ edges having both endpoints the same color.

(i) If G is 3-colorable (has a proper 3-coloring with 0 monochromatic edges), we can construct a valid coloring for G'. Use the proper 3-coloring for the subgraph G. For each of the k copies of K_4, use a 3-coloring. Since K_4 cannot be properly 3-colored, any 3-coloring must produce at least one monochromatic edge. It is possible to 3-color K_4 with exactly one monochromatic edge. Use such a coloring for each copy. The total number of monochromatic edges in G' is exactly $0 + k \times 1 = k$. Thus, G' is 3-colorable with exactly k monochromatic edges, satisfying the condition (at most k).

(ii) If G' has a 3-coloring with at most k monochromatic edges. As established, any 3-coloring of a K_4 subgraph requires at least one monochromatic edge. Since there are k disjoint copies of K_4 in G', the total number of monochromatic edges must be at least $k \times 1 = k$. Given that the total number of monochromatic edges is *at most* k, it must be that *exactly* k monochromatic edges exist, and all of them must reside within the K_4 copies. This implies that the original subgraph G must have 0 monochromatic edges in this coloring. Thus, the restriction of the coloring to G is a proper 3-coloring for G.

Runtime analysis: The reduction involves adding k small graphs (K_4). Since $k = 31337$ is a constant, this construction takes time polynomial in the size of G. Therefore, Relaxed 3-Coloring is NP-hard. (It's also in NP, thus NPC).

Undirected s-t Hamiltonian Path $\leq_P$ Bounded-Degree Spanning Tree (BDST)

Problem (BDST): Given an undirected graph $G = (V, E)$, does G contain a spanning tree in which every vertex has degree at most $D = 7$?

Goal: Show BDST (with $D = 7$) is NP-hard by reducing Undirected s-t Hamiltonian Path (which is NPC) to it.

Reduction from Undirected s-t Hamiltonian Path:

1. Given an instance $(G = (V, E), s, t)$ of the undirected s-t Hamiltonian path problem.
2. Create an instance G' of the 7-degree spanning tree problem: Start with G.
3. For each vertex $v \in V \setminus \{s, t\}$, add 5 new dummy vertices $d_{v1}, \ldots, d_{v5}$ to G' and connect them only to v with edges (v, d_{vi}).
4. For vertex s, add 6 new dummy vertices $d_{s1}, \ldots, d_{s6}$ and connect them only to s with edges (s, d_{si}).
5. For vertex t, add 6 new dummy vertices $d_{t1}, \ldots, d_{t6}$ and connect them only to t with edges (t, d_{ti}).
6. Ask: Does G' have a spanning tree in which every vertex has degree at most 7?

Proof of correctness: We show that G has an s-t Hamiltonian path if and only if G' has a 7-degree spanning tree.

(i) ($\Rightarrow$) If G has an s-t Hamiltonian path P. Construct a potential spanning tree T' in G' as follows: Include all edges of the path P. Include all edges connecting original vertices to their corresponding dummy vertices.

- T' is spanning: It includes all original vertices (via P) and all dummy vertices (via their connections).
- T' is connected: The path P connects all original vertices, and dummies are connected to them.
- T' is acyclic: P is acyclic, and adding the dummy "leaf" edges doesn't create cycles. Thus T' is a tree.
- Degrees in T':
  - Dummy vertex: degree 1 (≤ 7).
  - $v \in V \setminus \{s, t\}$: degree in P is 2. Degree in T' is $2 + 5 = 7$.
  - s: degree in P is 1. Degree in T' is $1 + 6 = 7$.
  - t: degree in P is 1. Degree in T' is $1 + 6 = 7$.
- All degrees are ≤ 7. Thus, G' has a 7-degree spanning tree.

(ii) ($\Leftarrow$) If G' has a spanning tree T' in which every vertex has degree at most 7.

- In T', each dummy vertex must be connected to its corresponding original vertex (this is its only possible connection).
- Consider an original vertex $v \in V \setminus \{s, t\}$. Its degree in T' is ≤ 7. Since 5 edges connect it to its dummies, its degree using only original edges (edges from E) in T' must be $\leq 7 - 5 = 2$.
- For s, its degree in T' is ≤ 7. Since 6 edges connect to its dummies, its degree using original edges must be $\leq 7 - 6 = 1$. Similarly for t.
- Let T_{orig} be the subgraph of T' consisting only of the original vertices V and the original edges E that are present in T'.
- T_{orig} must be connected (if not, T' wouldn't be connected since dummies only connect to one original vertex).
- T_{orig} must be acyclic (since it's a subgraph of the tree T').
- So, T_{orig} is a tree spanning V.
- In T_{orig}, the degrees are ≤ 2 for internal nodes and ≤ 1 for s, t. The only tree satisfying this is a simple path between the two degree-1 nodes, s and t.
- Since this path spans all original vertices V, it is an s-t Hamiltonian path in G.

Runtime analysis: Adding dummy vertices and edges takes time polynomial in $|V|$. The reduction is polynomial. Since s-t Hamiltonian Path is NP-complete, BDST (with D=7) is NP-hard. (It is also in NP, hence NPC).

Maximum Independent Set $\leq_P$ Maximum Pyramid

Problem (Maximum Pyramid): Given an undirected graph $G = (V, E)$, find the size of the largest pyramid. A pyramid is a set of vertices $\{v_1, \ldots, v_k\}$ (the base) plus another vertex u (the apex) such that the base $\{v_1, \ldots, v_k\}$ is an independent set in G, and the apex u is adjacent to every vertex v_i in the base. The size of the pyramid is $k + 1$.

Goal: Show the problem of finding the size of the largest pyramid is NP-hard by reducing Maximum Independent Set (MaxIS) to it.

Reduction from Maximum Independent Set:

1. Given an instance $G = (V, E)$ of the MaxIS problem.
2. Create an instance $G' = (V', E')$ of the maximum pyramid problem:
   - $V' = V \cup \{z\}$, where z is a new vertex not in V.
   - $E' = E \cup \{(z, v) \mid v \in V\}$. (Add z and connect it to all original vertices).
3. Find the size K of the largest pyramid in G'.
4. Output $K - 1$ as the size of the maximum independent set in G.

Proof of correctness: We show that MaxPyramidSize(G') = MaxISSize(G) + 1.

(i) ($\Rightarrow$) If G has a maximum independent set $S = \{v_1, \ldots, v_k\}$ of size k. Consider the set $P = S \cup \{z\}$ in G'. The base S is independent in G, and since no edges were added between vertices in V, S is also independent in G'. The apex z is connected to all $v_i \in S$ by construction of G'. Thus, P is a valid pyramid in G' of size $k + 1$. This implies MaxPyramidSize(G') $\geq k + 1 =$ MaxISSize(G) + 1.

(ii) ($\Leftarrow$) If G' has a maximum pyramid $P = \{v_1, \ldots, v_k\} \cup \{u\}$ of size $K = k + 1$. The base $S = \{v_1, \ldots, v_k\}$ is, by definition, an independent set in G'. Since the apex u must be connected to all v_i, none of the v_i can be the special vertex z (as z has no self-loops). Therefore, $S \subseteq V$. Since the only edges added to G' involved z, the independence of S holds also in G. Thus, G has an independent set S of size $k = K - 1$. This implies MaxISSize(G) $\geq K - 1 =$ MaxPyramidSize(G') - 1.

Combining (i) and (ii), we have MaxPyramidSize(G') = MaxISSize(G) + 1.

Runtime analysis: The reduction (adding vertex z and $|V|$ edges) takes polynomial time. Since MaxIS is NP-hard, the Maximum Pyramid problem is also NP-hard. (The corresponding decision problem is NPC).